A PROJECT REPORT ON

OIL SLICK DETECTION AND DRIFT PREDICTION USING SATELLITE IMAGERY

SUBMITTED TO THE SAVITRIBAI PHULE PUNE UNIVERSITY, PUNE

IN THE FULFILLMENT FOR THE AWARD OF THE DEGREE

BACHELOR OF ENGINEERING IN INFORMATION TECHNOLOGY

BY

Paras Ningune B400070520

Sakshi Patil B400070523

Soham Mane B400070516

Siddhant Pote B400070525

UNDER THE GUIDANCE OF

Prof. M. R. Apsangi

DEPARTMENT OF INFORMATION TECHNOLOGY

Pune Vidyarthi Griha's College of Engineering, Technology and Management

Pune – 411009

Academic Year: 2025–26

CERTIFICATE

This is to certify that the project report entitled

OIL SLICK DETECTION AND DRIFT PREDICTION USING SATELLITE IMAGERY

Submitted by

Paras Ningune B400070520
Sakshi Patil B400070523
Soham Mane B400070516
Siddhant Pote B400070525

is a bonafide work carried out by them under the supervision of **Prof.M.R.Apsangi** and is approved for the fulfillment of the requirement of Savitribai Phule Pune University for the award of the Degree of Bachelor of Engineering (Information Technology).

This project report has not been earlier submitted to any other Institute or University for the award of any degree or diploma.

Prof.M.R.Apsangi	**Dr.S.A.Mahajan**
Internal Guide	Head of Department
Department of Information Technology	Department of Information Technology
External Examiner	**Dr. S.R. Patil**
	Principal
	PVG's COETM, Pune

Date: _____

Place: Pune

ACKNOWLEDGEMENT

We take this opportunity to thank our project guide **Prof. M. R. Apsangi** and Head of the Department **Dr. S. A. Mahajan** for their valuable guidance and for providing all the necessary facilities, which were indispensable in the completion of this project report. We are also thankful to all the staff members of the **Department of Information Technology of Pune Vidyarthi Griha's College of Engineering and Technology** for their valuable time, support, comments, suggestions and persuasion. We would also like to thank the institute for providing the required facilities, Internet access and important books. Finally, we thank our friends and family for their moral support.

Paras Ningune
Sakshi Patil
Soham Mane
Siddhant Pote

LIST OF FIGURES

LIST OF TABLES

NOMENCLATURE

Symbol/Term	Description
CNN	designed to process and analyze grid-structured data
Epoch	One complete cycle through the training dataset
ResNet	A deep neural network with residual connections for efficient image classification.
Eulerian approach	A method that analyzes changes at fixed locations in a fluid flow field.
Lagrangian Model	A method that tracks the movement of individual particles in a flow field.
MAE	Mean Absolute Error, measures average error between images
U-Net	A convolutional neural network used for image segmentation.
SegNet	deep learning architecture designed for semantic image segmentation.
Polarization	The orientation of electromagnetic wave oscillations used to capture different surface characteristics
Normalization	Scaling image data to a standard range for model input
Segmentation	Process of partitioning an image into meaningful regions

CONTENTS

ABSTRACT

Marine oil spills pose a significant threat to marine ecosystems, coastal environments, and maritime activities, making their timely detection and monitoring essential for effective response and mitigation. This work presents an intelligent framework for oil slick detection and drift prediction using Sentinel-1 Synthetic Aperture Radar (SAR) imagery. The proposed system automates the process of identifying oil spills, accurately segmenting the affected regions, and forecasting their future movement under varying environmental conditions.

The acquired SAR images are preprocessed to reduce speckle noise and enhance image quality before being analyzed using deep learning models. A ResNet-based classification model is employed to distinguish oil slicks from non-oil regions and look-alike phenomena, while a U-Net segmentation model is used to generate precise oil slick masks. The geographical coordinates extracted from the segmented oil slick are then utilized by the OpenDrift framework to perform Lagrangian particle-based drift simulation using environmental parameters such as ocean currents and wind conditions.

Experimental results demonstrate that the proposed approach achieves accurate oil slick detection and reliable drift prediction, generating trajectory maps and animations that can support environmental agencies and maritime authorities in spill response and containment planning. The developed framework provides an end-to-end solution for satellite-based marine oil spill monitoring and can be extended for real-time environmental surveillance applications.

CHAPTER 1

INTRODUCTION

Oil slicks-thin layers of petroleum floating on the ocean surface-have become a critical environmental concern due to their rapid spread and severe ecological consequences. While some oil reaches the surface through natural seepage from the seafloor, most slicks originate from human activities such as tanker accidents, pipeline failures, offshore drilling incidents, and improper disposal of ship waste. Even a small quantity of oil can expand over vast oceanic areas, blocking sunlight and oxygen exchange essential for microscopic phytoplankton, which form the foundation of the marine food chain. Marine organisms may ingest oil directly or through contaminated prey, while seabirds lose their waterproofing ability, often resulting in irreversible harm. These cascading effects highlight the urgent need for efficient detection and monitoring of oil spills to protect marine ecosystems.

Traditional approaches to oil spill monitoring relied heavily on ship patrols and aerial surveys. Although useful, these methods are expensive, resource-intensive, and severely limited in spatial coverage. Ships can only inspect narrow regions at a time, requiring large crews, high fuel consumption, and extensive planning. Aerial surveys, while faster, rely on skilled personnel, specialized aircraft, and favourable weather conditions. Most importantly, both methods observe only specific locations at specific times, making it impossible to maintain continuous watch over dynamic ocean environments. As oil slicks disperse rapidly due to winds and currents, delays in detection can significantly increase the environmental and economic losses associated with spill events.

The advent of satellite-based observation has transformed the field of marine oil spill monitoring by overcoming the limitations of conventional surveillance. Satellites provide large-scale, repeated coverage of the world's oceans, including remote or inaccessible regions. They can operate under cloudy, stormy, or nighttime conditions, making them indispensable for early detection. Real-time or near-real-time data enables authorities to respond faster, issue public alerts, and coordinate cleanup operations more effectively. Continuous monitoring also allows experts to study the evolution and drift of slicks over long periods, improving the accuracy of forecasting and facilitating quick mitigation actions.

Table 1.1: Comparison of satellite sensors and their characteristics (Adapted from [26])

Satellite	Sensor Type	Spatial Resolution	Revisit Time
Sentinel-1	C-Band SAR	~10 m	3–4 days
Sentinel-2	Multispectral Optical	10–60 m	5 days
RADARSAT	C-Band SAR	3–100 m	24 hrs
TerraSAR-X	X-band SAR	~1–3 m	Depends on orbit & tasking
RISAT-2	X-band SAR	~1 m	Depends on orbit & tasking
Cartosat-2	Panchromatic optical	~1–3 m	5 days

Among satellite instruments, Synthetic Aperture Radar (SAR) has become one of the most powerful tools in oil spill detection. SAR sensors such as those aboard the Sentinel-1 mission generate high-resolution images irrespective of weather or illumination conditions. Oil slicks appear as dark formations in most SAR imagery because they dampen the small surface waves responsible for radar backscatter. However, natural phenomena like biogenic films, low-wind areas, or organic debris can resemble oil spills, complicating detection. Early detection methods relied on simple thresholding or manual visual inspection, while later approaches adopted machine learning models like Support Vector Machines and Random Forests [23]. Today, deep learning architectures including CNNs, U-Net, and SegNet dominate the field, achieving higher accuracy but still facing challenges such as false positives and limited availability of labeled training data.

Understanding where an oil slick will move after detection is equally crucial. Drift prediction models seek to estimate the future trajectory of oil patches by incorporating environmental variables such as winds, surface currents, wave fields, spreading behaviour, evaporation, and chemical breakdown of oil. Lagrangian models track individual oil particles, while Eulerian models estimate concentration changes over spatial grids. Integrating SAR imagery with hydrodynamic simulations and in-situ measurements from buoys or coastal sensors significantly enhances prediction reliability [20] during real-world crisis operations [19]. This creates an end-to-end system in which satellites detect the spill, algorithms identify it, and drift models forecast its movement—allowing response teams to strategically deploy containment equipment.

A key component in refining detection accuracy is the use of different SAR polarization modes, particularly those employed by Sentinel-1. Polarization refers to the orientation of radar waves during transmission and reception—either horizontal (H) or vertical (V). Single-polarization modes (VV or HH) capture co-polarized signals and are simple

to process but provide limited surface information. Dual-polarization modes, such as VH or HV, transmit in one polarization and receive in both co- and cross-polarized channels. These richer datasets reveal detailed scattering properties of the sea surface, enabling superior classification performance. For oil spill detection, dual-polarization SAR generally enhances reliability by distinguishing true oil slicks from look-alike phenomena.

Table 1.2: Sentinel-1 Modes

Polarization	**Meaning**	**Modes**
VV	Transmit Vertical, Receive Vertical	Single Polarization
HH	Transmit Horizontal, Receive Horizontal	Single Polarization
HV	Transmit Horizontal, Receive Vertical	Dual Polarization (HH + HV)
VH	Transmit Vertical, Receive Horizontal	Dual Polarization (VV + VH)

Overall, the integration of advanced satellite technologies, improved SAR imaging capabilities, and robust computational models has significantly strengthened global oil spill monitoring and response efforts. Despite persistent challenges—such as look-alike surface features, rapidly changing environmental conditions, and the ongoing need for larger labeled datasets—the field continues to evolve. Future advancements may emerge from multi-sensor fusion (combining drones, satellites, and ground systems), hybrid human-AI decision frameworks, and next-generation deep learning models. As these technologies mature, they offer the promise of more accurate detection, faster response, and ultimately, better protection of vulnerable marine ecosystems.

1.1 AIM

To develop system that detects oil slicks on satellite SAR imagery using deep learning and predicts their drift using environmental data, enabling faster and more accurate monitoring and decision-making for marine pollution response

1.2 MOTIVATION

The motivation for this project arises from the urgent need to detect and manage oil spills quickly to reduce environmental damage. Oil slicks spread rapidly and can harm marine life, fisheries, and coastal ecosystems, making early identification critical. Traditional monitoring methods are slow, costly, and limited by weather and visibility, leading to delays in response. Using SAR satellite imagery combined with deep learning enables faster, more accurate detection of oil slicks without being affected by clouds, lighting, or distance. Additionally, predicting the drift of detected slicks helps authorities plan containment measures in advance, reducing the area affected by the spill. This project aims to create an automated system that improves monitoring efficiency, supports informed decision-making, and strengthens environmental protection efforts.

1.3 OBJECTIVES

- Apply image processing and deep learning to reduce false alarms
- Improving the prediction accuracy and generalizing the model
- Integrate environmental data (currents, wind, tides) for drift prediction
- Forecast slick movement and impacted areas through predictive modeling
- Develop a unified, scalable system for real-time monitoring and response

CHAPTER 2
LITERATURE SURVEY

Detection of oil spills in SAR satellite images using the Otsu-Bradley thresholding method [1] focuses on automatically identifying oil-contaminated regions on the ocean surface. The proposed approach combines two thresholding techniques: the global thresholding of Otsu and Bradley's local adaptive thresholding. The Otsu method separates potential slick areas from water based on the overall image intensity, while the Bradley method refines the segmentation by adapting to local brightness variations. This hybrid approach improves detection accuracy, reduces false positives caused by calm water or natural slicks, and does not require training data. The method is computationally efficient and suitable for fast real-time oil slick detection in remote sensing applications.

Textural Features for Image Classification[2] focuses on using texture as a measurable property for image analysis and classification. The study introduces the Gray-Level Co-occurrence Matrix (GLCM)[10], which captures how often different pixel intensity pairs occur at specific spatial distances and directions in an image. From this matrix multiple statistical texture descriptors are derived, such as contrast, correlation, energy, and homogeneity.The paper demonstrates that these textural descriptors help differentiate image regions that may have similar intensity but different surface patterns. This makes texture-based classification more reliable in applications such as remote sensing [24], medical imaging, and pattern recognition. The contribution of this work established GLCM as one of the standard approaches to texture feature extraction, which has been widely adopted in fields like oil spill detection [24], improving the accuracy and robustness of image classification tasks.

Oil Spill Detection using Convolutional Neural Networks[3] and Sentinel-1 SAR Imagery focuses on using deep learning to automatically identify oil slicks from satellite SAR images. Sentinel-1 provides radar images that capture the ocean surface in any weather or lighting conditions, making it suitable for real-time monitoring. A Convolutional Neural Network (CNN) is trained on labeled SAR image patches that contain oil spills and non-oil areas. The model learns to recognize patterns and textural differences between actual oil slicks and look-alike features such as calm water or natural films. By extracting spatial features directly from the imagery, the CNN eliminates the need for hand-crafted feature design. The study shows that CNN-based detection achieves higher accuracy and fewer false positives compared to traditional

image-processing or thresholding methods. It also improves detection efficiency by reducing manual analysis time, making it useful for large-scale oil spill monitoring and emergency response.

Dark Spot Detection in SAR Images of Oil Spill Using SegNet[4] focuses on detecting oil slicks (dark spots) in SAR satellite imagery using a deep learning segmentation model. The study applies the SegNet architecture, which is an encoder-decoder CNN designed for pixel-wise image segmentation. The encoder extracts spatial and textural features from the SAR image, while the decoder reconstructs a segmented map where dark regions corresponding to oil slicks are highlighted. This automated segmentation reduces the need for manual inspection and minimizes false detections caused by look-alike dark patterns such as calm water or natural slicks. The method improves accuracy and consistency by learning complex spatial characteristics of oil slicks directly from data. It is computationally efficient and suitable for large-scale satellite image processing, making it valuable for oil spill monitoring and rapid response systems.

Convolutional Networks for Biomedical Image Segmentation [5] introduces a deep learning model specifically designed for image segmentation tasks in biomedical applications. The proposed U-Net architecture is based on an encoder-decoder convolutional neural network with skip connections that preserve spatial information lost during feature extraction. The encoder extracts high-level features from the input image, while the decoder reconstructs a pixel-wise segmentation map. This architecture achieves high segmentation accuracy even with limited training data and has become one of the most widely adopted deep learning models for semantic image segmentation.

A Generic Framework for Trajectory Modelling [12] presents an open-source framework for simulating the movement of particles in marine and atmospheric environments. The proposed framework, OpenDrift v1.0, uses real environmental data such as ocean currents, wind, and wave conditions to model the transport of floating objects. Its modular architecture allows adaptation to a wide range of applications, including oil spill drift prediction, drifting debris, algae transport, and search-and-rescue operations. The framework provides ready-to-use models for oil spill simulation and enables researchers and environmental agencies to perform both research-oriented and real-time trajectory forecasting. The paper demonstrates that OpenDrift provides a flexible, efficient, and data-driven platform for analysing and predicting particle movement.

A Numerical Oil Spill Model Based on a Hybrid Method[7] presents an approach that combines two different modelling techniques to simulate the movement and spreading of oil spills on the ocean surface. The model uses both Lagrangian and Eulerian methods: the Lagrangian part tracks individual oil particles as they move with currents

and wind, while the Eulerian method estimates how the oil concentration spreads and changes over a grid. By combining both methods, the model captures detailed particle movement as well as large-scale distribution of oil slicks. It considers important environmental factors such as wind, waves, ocean currents, and oil weathering processes like evaporation and dispersion. This hybrid approach improves prediction accuracy, especially in complex ocean conditions, and helps authorities understand where the oil will drift and how it will spread over time. The paper demonstrates that hybrid modelling provides a more robust and realistic simulation compared to using only one of the methods.

Influence of Sea Surface Waves on Numerical Modeling of an Oil Spill[8] examines how ocean surface waves affect the movement and behavior of oil slicks during numerical simulations. Traditional oil spill models often focus on wind and ocean currents, but this paper highlights that waves play a crucial role in determining how oil spreads on the water. Waves generate additional surface motion, known as wave-induced drift or Stokes drift, which pushes oil in different directions than currents alone would suggest. The study integrates wave data into the oil spill model and shows that including wave effects significantly improves the accuracy of trajectory predictions. When waves are ignored, the predicted oil slick position can deviate from the actual drift path, leading to incorrect planning of response operations. The paper concludes that incorporating sea surface wave dynamics into oil spill modelling results in more realistic simulations and better decision-making during spill management and containment efforts.

The key findings, methodologies, and limitations of the literature surveyed are summarized in Table 2.1

Table 2.1: Literature Survey

Sr. No.	**Reference Name**	**Description**	**Problems**
1	Detection of Oil Spills in SAR Satellite Images using the Otsu-Bradley Thresholding Method	Uses a hybrid thresholding technique by combining Otsu's global thresholding with Bradley's adaptive thresholding for automatic oil spill segmentation in SAR images.	Sensitive to image noise and varying sea conditions. Unable to accurately distinguish oil spills from look-alike features in complex environments.

Sr. No.	Reference Name	Description	Problems
2	Textural Features for Image Classification	Introduces the Gray-Level Co-occurrence Matrix (GLCM) for extracting texture features such as contrast, correlation, energy, and homogeneity to improve image classification.	Requires manual feature extraction and parameter tuning. Performance decreases when texture patterns are highly complex.
3	Oil Spill Detection using Convolutional Neural Networks and Sentinel-1 SAR Imagery	Applies a CNN to classify Sentinel-1 SAR image patches into oil spill and non-oil categories by automatically learning spatial features.	Requires a large labeled dataset and significant computational resources for training. May not provide pixel-level localization.
4	Dark Spot Detection in SAR Images of Oil Spill Using SegNet	Employs the SegNet encoder-decoder architecture for semantic segmentation of oil slicks in SAR imagery.	High computational cost and segmentation quality depends heavily on the availability of annotated training data.
5	U-Net: Convolutional Networks for Biomedical Image Segmentation	Introduces a U-Net architecture with skip connections to perform accurate pixel-wise image segmentation while preserving spatial information.	Originally designed for biomedical images and requires adaptation and retraining for SAR image segmentation tasks.
6	OpenDrift v1.0: A Generic Framework for Trajectory Modelling	Provides an open-source framework for predicting oil spill drift using wind, ocean current, and wave data.	Prediction accuracy depends on the availability and precision of environmental datasets.

Sr. No.	**Reference Name**	**Description**	**Problems**
7	A Numerical Oil Spill Model Based on a Hybrid Method	Combines Lagrangian particle tracking and Eulerian concentration models to simulate oil spill movement and spreading.	Computationally intensive and requires accurate environmental parameters for reliable predictions.
8	Influence of Sea Surface Waves on Numerical Modeling of an Oil Spill	Studies the effect of wave-induced drift (Stokes drift) on oil spill trajectory prediction and integrates wave dynamics into numerical models.	Requires high-quality wave data and increases model complexity and computation time.

CHAPTER 3
PROBLEM STATEMENT

To develop a deep learning-based system that automatically detects oil slicks from SAR satellite imagery and predicts their drift, enabling faster response while reducing manual effort and misclassification due to look-alike dark regions.

CHAPTER 4
SOFTWARE REQUIREMENTS AND SPECIFICATION

4.1 PROBLEM DEFINITION

Satellite-based monitoring of marine environments often requires manual inspection to identify oil slicks and estimate their drift direction. This process is slow, prone to human errors, and not suitable for real-time response in emergency situations. Radar satellite images contain complex textures and noise, making traditional image processing methods inaccurate. A system is required that automatically detects oil slicks from SAR images and predicts their drift using environmental and oceanographic data. The goal is to provide accurate oil spill detection and future movement estimation to support faster decision-making and reduce environmental damage.

4.2 FUNCTIONAL REQUIREMENTS

- SAR Image Upload : The system shall allow users to upload satellite SAR images.
- Dark Spot Detection : The system shall automatically detect and segment oil slick regions using a deep learning model (SegNet/U-Net).
- Drift Prediction : The system shall estimate future drift trajectory using numerical modeling frameworks
- Environmental Data Integration : The system shall incorporate wind, current, and wave direction data from APIs to improve drift accuracy.
- Output Visualization : The system shall display detected oil slick regions and predicted drift paths on a map interface.

4.3 NON-FUNCTIONAL REQUIREMENTS

- Performance: Oil slick detection and drift simulation must complete within an acceptable time.
- Security: Remote sensing data and API keys must be securely stored.
- Usability: The interface should be intuitive for researchers and authorities.
- Scalability: The system should handle large datasets and multiple image uploads.
- Maintainability: System must support updates to models and API integrations.

4.4 SOFTWARE AND HARDWARE REQUIREMENTS

- Software: Python(Tensorflow, Scikit-learn, OpenCV, RasterIO), Ubuntu, SNAP, QGIS and OpenDrift.
- Hardware: Standard workstation with GPU support. (Minimum RTX4050).
- Data: Dataset: Sentinel-1 SAR Oil spill image dataset. Weather Data: OpenWeatherMap API

4.5 USE CASES

- UC1: Fetch Satellite Data
- UC2: Image Preprocessing
- UC3: Run Detection Model
- UC4: Fetch Environmental Data
- UC5: Drift Trajectory Prediction
- UC4: Generate Spill Map

CHAPTER 5
FLOWCHART

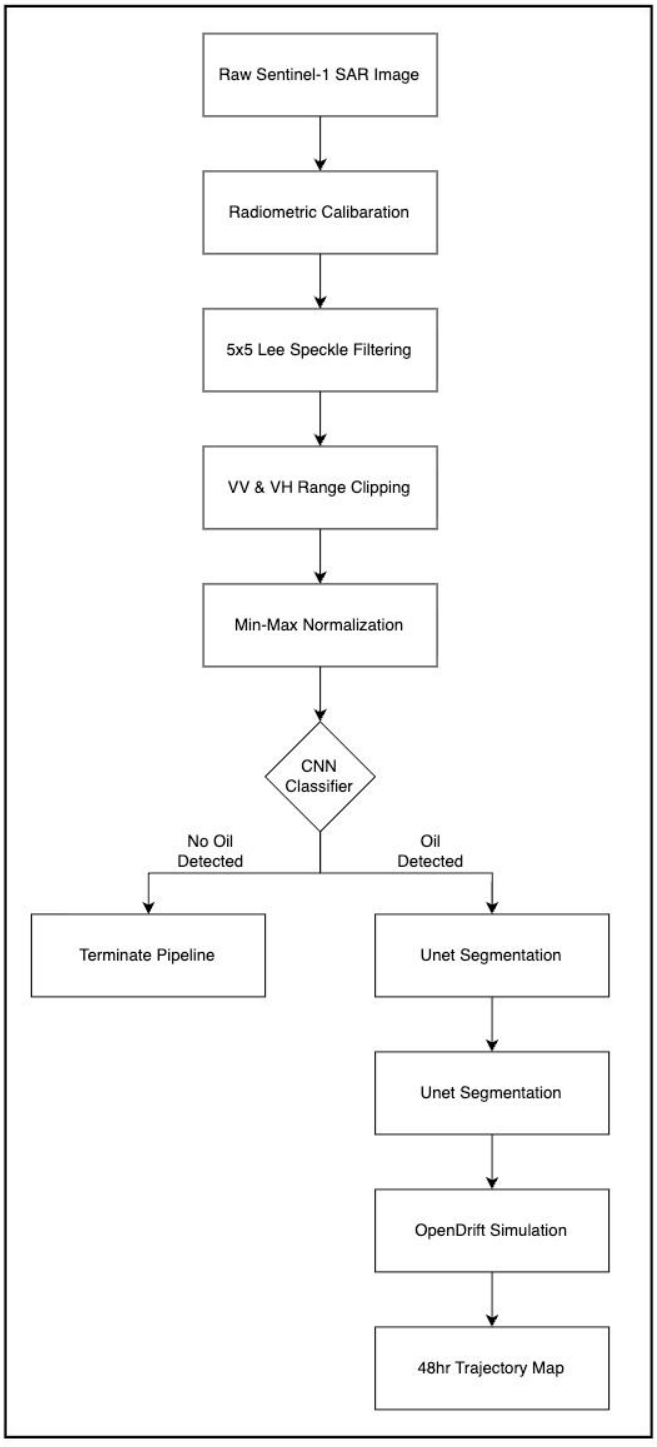


Figure 5.1: Flow Chart

CHAPTER 6
PROJECT REQUIREMENTS SPECIFICATION

6.1 METHODOLOGY

6.1.1 Data Collection

The plan is to acquire high-resolution Synthetic Aperture Radar (SAR) images from publicly available repositories such as Copernicus Open Access Hub (Sentinel-1 data). These images should cover diverse oceanic regions, seasons, and weather conditions to ensure generalization. SAR imagery alone only tells part of the story. Environmental datasets covering wind speed and direction, ocean currents, tidal patterns, and wave height are pulled in from CMEMS (Copernicus Marine Environment Monitoring Service) to fill in the gaps. Once collected, all datasets are carefully annotated to highlight oil slick regions, creating the labeled foundation needed for supervised deep learning-based classification.

6.1.2 Image Preprocessing

Raw Sentinel-1 SAR images require thorough preprocessing before any meaningful analysis can be conducted. This begins with radiometric calibration, followed by speckle noise filtering using established techniques such as Lee or Frost filters, and geometric correction to ensure spatial consistency across all acquired scenes. Image intensities are subsequently normalized, and each image is cropped or resampled to a uniform resolution of 256×256 pixels to provide the model with consistently structured input. For the purpose of model training, oil and non-oil regions are delineated either manually or through semi-automatic labeling methods, producing the segmentation masks required for supervised learning. On the environmental side, datasets encompassing wind speed and direction, ocean currents, tidal patterns, and wave height are systematically cleaned and temporally synchronized with the acquisition timestamps of their corresponding SAR images ensuring that the environmental context presented to the model accurately reflects the oceanographic conditions at the precise moment each image was captured.

6.1.3 Slick Detection and Segmentation Model

A Convolutional Neural Network (CNN)-based deep learning architecture is employed for the automatic detection of oil slicks from SAR imagery. The model is trained to extract and learn complex spatial and contextual features directly from the radar backscatter patterns present in each image. This capability is particularly critical given the inherent challenge of distinguishing true oil slicks from visually similar phenomena — commonly referred to as look-alikes (such as low wind zones, biogenic surface films, and other natural oceanic features) that can produce comparable signatures in SAR data.

Detection Model:

- The dataset was divided into training (66%) and validation (34%) sets.
- A separate test set consisting of 450 images was created for evaluation.
- The model was optimized using binary cross-entropy loss in conjunction with the Adam optimizer.
- Model performance was evaluated using metrics such as Accuracy, Precision, Recall, and F1-score.

Segmentation Model:

- Trained a U-Net model to generate accurate oil slick segmentation masks, drawing inspiration from advanced architectures like the Dual Attention U-Net [11].
- Fine-tuned hyperparameters such as learning rate, kernel size, and dropout rate to achieve optimal performance.
- Evaluated model predictions against ground truth masks using Intersection over Union (IoU).

Extract key features such as backscatter intensity, texture metrics [22], and shape descriptors to improve model discriminability. Augment the dataset using transformations such as rotation, scaling, flipping, and brightness variations to enhance robustness. Statistical descriptors of wind, tide, and current data are also encoded as auxiliary input features to assist in drift modeling.

6.1.4 Drift Prediction Model

Following the detection and segmentation of the oil slick, the system predicts its future movement using a Lagrangian particle-based drift model. The oil slick mask generated by the U-Net segmentation model is used to identify the spill location, while

the corresponding geographical coordinates are extracted from the GeoTIFF metadata. These coordinates are used to initialize virtual particles representing the detected oil slick.

The drift prediction process incorporates environmental factors such as:

- Surface ocean currents
- Wind velocity and direction
- Wave-induced transport
- Turbulent diffusion

These environmental forcings are essential for evaluating drift trajectory, though they introduce uncertainties that can be evaluated via ensemble techniques [18].The simulation is performed using the OpenDrift framework, which models the movement of oil particles under the influence of these environmental conditions. The position of each particle is updated according to:

$$X_{t+1} = X_t + (V_c + V_w)\Delta t$$

where:

X_t is current particle position

V_c is ocean current velocity

V_w is wind-induced velocity

Δt is simulation time step

The model forecasts the trajectory of the oil slick over a predefined period of 24–48 hours and generates visualizations of the predicted spill movement.

The final output includes:

- Predicted oil slick trajectory
- Drift animation
- Geographical visualization of particle movement
- Estimated affected region

The generated forecasts assist environmental agencies and disaster-response teams in planning containment and cleanup operations by providing an estimate of the future spread of the detected oil spill.

6.1.5 System Integration and Visualization

Integrate the detection and drift prediction modules into a unified system pipeline. Develop a web-based interface using Python frameworks like Flask for backend and React.js for frontend. Allow users to upload SAR images, view detected oil regions, oil spill area and visualize predicted drift paths on an interactive map (via Plotly). Provide export functionality for reports and trajectory maps.

6.2 ALGORITHMS AND TECHNOLOGIES

6.2.1 Algorithms

Convolutional Neural Network (CNN):

- **Purpose:** Automatically identify and classify oil slick regions in SAR imagery.
- **Explanation:** Learns hierarchical spatial and textural features from input images, enabling differentiation between oil spills and look-alike phenomena such as low-wind areas or natural films.
- **Application:** Trained on labeled Sentinel-1 SAR datasets to perform oil spill detection.

U-Net:

- **Purpose:** Perform precise pixel-wise segmentation of oil slick regions in SAR images.
- **Explanation:** Utilizes an encoder–decoder architecture with skip connections to preserve spatial details while capturing contextual information, resulting in accurate boundary detection.
- **Application:** Applied to Sentinel-1 SAR data to generate binary masks representing oil-contaminated areas.

Lagrangian Drift Model:

- **Purpose:** Simulate the movement and trajectory of oil particles over time.
- **Explanation:** Tracks individual particles influenced by environmental forces such as ocean currents, wind, and wave-induced drift.
- **Application:** Used for short-term prediction of oil spill movement and trajectory analysis.

6.2.2 Technologies

- **Python:** Primary programming language used for model development, data processing, and system integration.
- **TensorFlow:** Deep learning framework used for designing and training CNN and U-Net models.
- **OpenCV, RasterIO, and NumPy:** Utilized for image preprocessing, geospatial data handling, and numerical computations.
- **Flask:** Lightweight web framework used to develop the backend and integrate model outputs into a user interface.
- **CMEMS APIs:** Provide real-time oceanographic and meteorological data (e.g., currents, wind) for drift modeling.
- **Matplotlib and Plotly:** Used for visualization of oil slick detection results and drift trajectory simulations.

CHAPTER 7
SYSTEM ARCHITECTURE

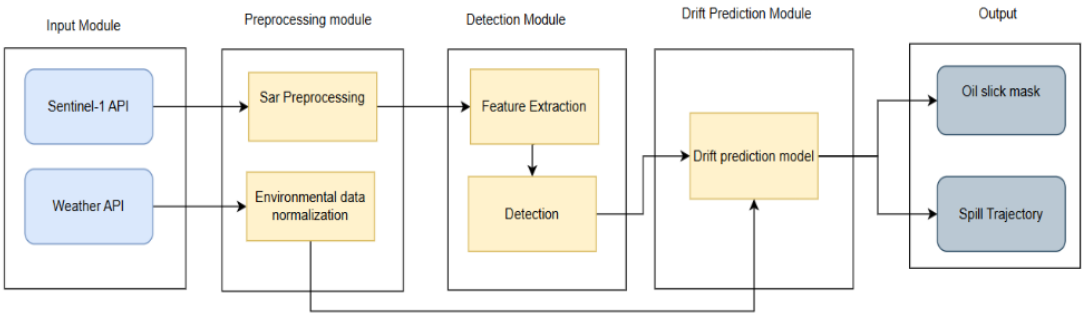


Figure 7.1: System Architecture of Oil Slick Detection and Drift Prediction

Figure 7.1 illustrates the overall architecture of the proposed Oil Slick Detection and Drift Prediction system. The workflow begins with the Input Module, where Sentinel-1 SAR imagery is provided as input for oil slick analysis. The SAR image is then passed to the Preprocessing Module, where noise reduction, normalization, and image enhancement are performed to improve data quality before analysis.

The preprocessed image is forwarded to the Detection Module, where a ResNet-based classification model first determines whether the image contains an oil slick. If an oil slick is detected, a U-Net segmentation model generates a pixel-wise oil slick mask representing the affected region. The segmented oil slick and its geographical coordinates extracted from the GeoTIFF image are then provided to the Drift Prediction Module.

The Drift Prediction Module employs the OpenDrift framework to simulate the future movement of the detected oil slick using a Lagrangian particle-based approach. The simulation incorporates environmental forcing data such as ocean currents and wind conditions to forecast the trajectory and spread of the oil spill over a specified time period.

Finally, the Output Module presents the generated oil slick segmentation mask together with the predicted spill trajectory and drift visualization. This integrated architecture provides an end-to-end solution for automated oil spill detection, monitoring, and drift prediction, enabling timely decision-making for marine pollution response and environmental protection.

CHAPTER 8
HIGH LEVEL DESIGN

8.1 DATA FLOW DIAGRAMS

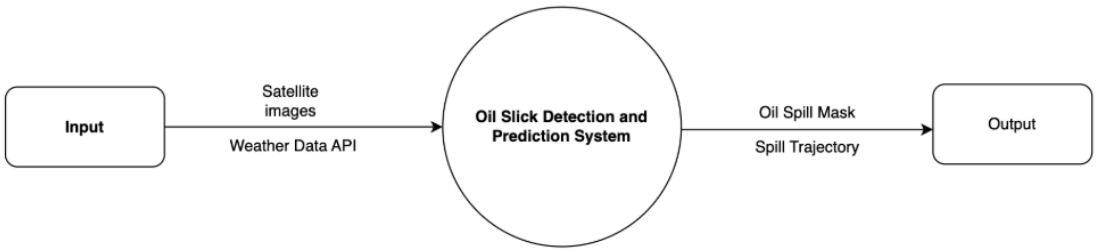


Figure 8.1: Data Flow Diagram - Level 0

The Level 0 Data Flow Diagram provides a high-level overview of the Oil Slick Detection and Prediction System. It shows how input data from satellite imagery and weather APIs enters the system and is processed to generate meaningful outputs. The system receives SAR images and environmental data, which are then analyzed by the central processing unit represented as a single process. Based on this analysis, the system produces two key outputs: an oil spill mask, which highlights detected spill regions, and a spill trajectory, which predicts the future movement of the oil slick. This top-level view simplifies the entire workflow into one main process, focusing only on the major inputs and outputs.

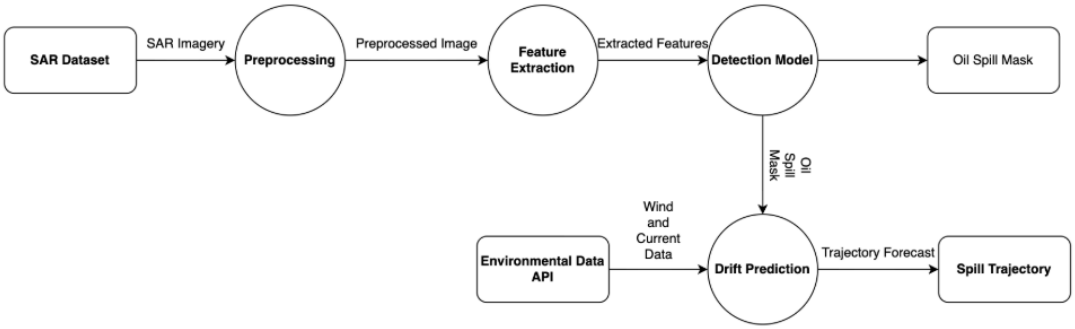


Figure 8.2: Data Flow Diagram - Level 1

The Level 1 Data Flow Diagram expands the internal processes of the system in detail. It begins with the SAR dataset, which flows into the Preprocessing module where the raw radar image is cleaned and normalized. The preprocessed image is then passed to the Feature Extraction module to identify patterns relevant to oil spills. These extracted features are fed into the Detection Model, which generates the oil spill mask. Simultaneously, environmental data such as wind and currents, obtained through the Environmental Data API, are used by the Drift Prediction module. This module combines the detected oil mask with environmental conditions to forecast the spill trajectory. The outputs oil spill mask and spill trajectory represenst the final processed results of the system.

8.2 UML DIAGRAMS

8.2.1 Use Case Diagram

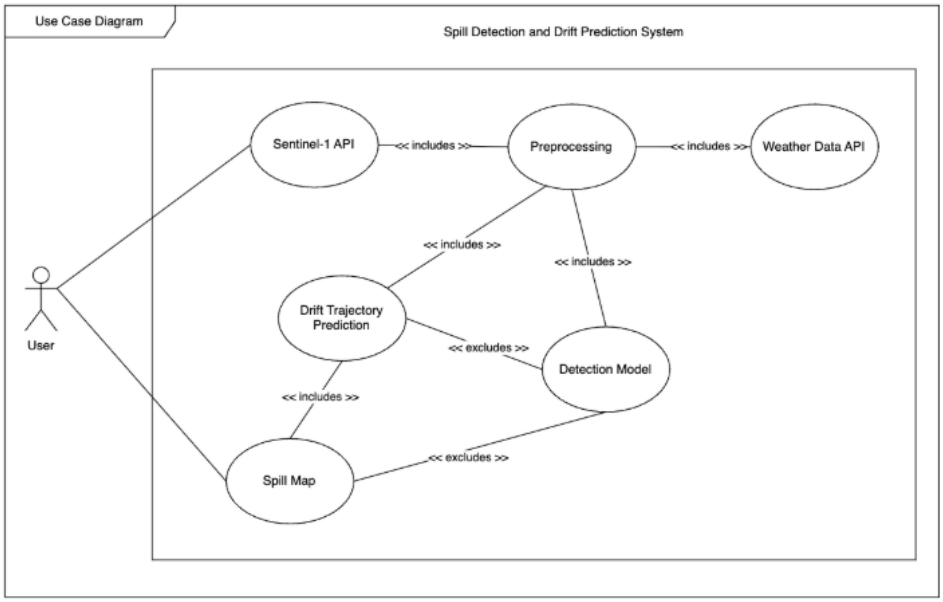


Figure 8.3: Use Case Diagram

This use case diagram represents how a user interacts with a Spill Detection and Drift Prediction System. The process begins when the user initiates the action to acquire SAR Image, which automatically includes steps like Preprocessing and Fetching Weather Data to prepare both satellite imagery and environmental conditions for analysis. Preprocessing ensures that the SAR image is corrected and enhanced, while weather data provides crucial inputs for later prediction stages.

Once preprocessing is completed, the Detection Model analyses the processed SAR image to identify the presence of an oil slick and generate the corresponding segmentation mask. The output of the Detection Model is then provided to the Drift Trajectory Prediction module, where environmental data obtained from the Weather Data API are incorporated to simulate the future movement of the detected oil spill. Thus, the drift prediction process is dependent on the successful detection and localization of the oil slick.

Finally, the system generates a Spill Map, which is a visual representation of detected oil regions and predicted drift paths. Generating the spill map includes trajectory prediction but excludes direct interaction with the detection model. Overall, the diagram shows how different system components work together, with clear includes and excludes relationships defining which processes are essential and which are optional within the workflow.

8.2.2 Class Diagram

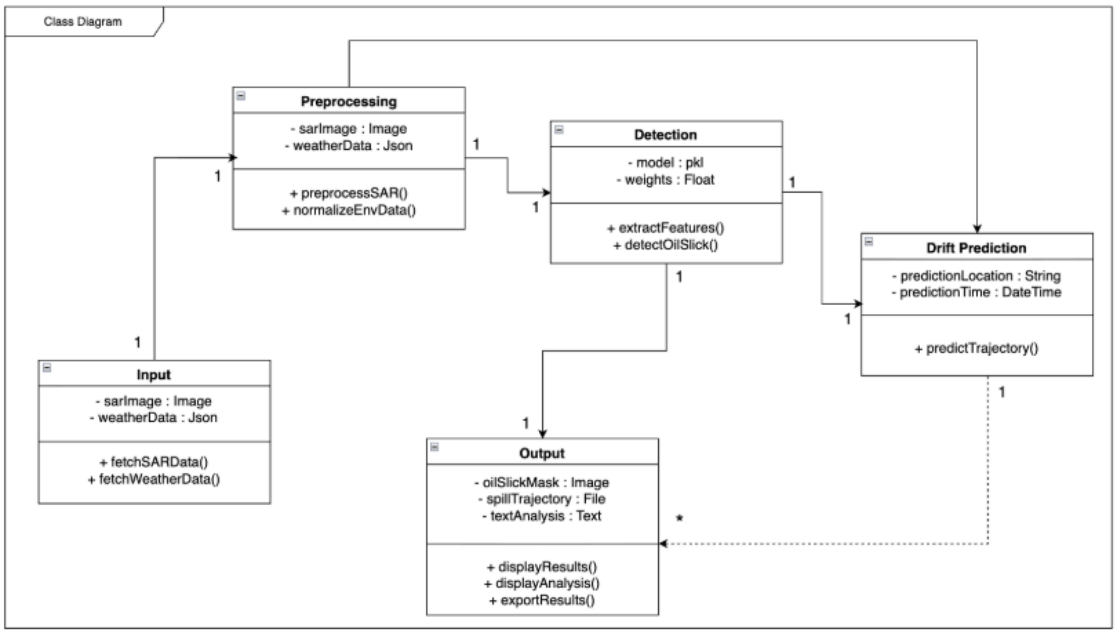


Figure 8.4: Class Diagram

This class diagram illustrates the structure and interaction between components in the Spill Detection and Drift Prediction System. The Input class collects raw SAR imagery and weather data and provides methods like fetchSARData() and fetchWeatherData() to retrieve them. This data flows into the Preprocessing class, which stores both inputs and performs essential operations such as preprocessSAR() and normalizeEnvData() to prepare the data for analysis.

The processed data is then passed to the Detection class, which contains the model and its parameters. This class handles core analytical functions including extractFeatures() and detectOilSlick(), producing a refined representation of oil slick regions. These outputs are sent to the Drift Prediction class, which uses attributes like prediction location and time to run the predictTrajectory() method and forecast how the oil slick will move.

Finally, the Output class compiles results from detection and prediction into visual and textual formats, including the oil slick mask, spill trajectory file, and summary analysis. It offers functions like displayResults() and exportResults() to present the findings to the user. The associations between classes show a clear, sequential flow where each component contributes to generating accurate and actionable output.

8.2.3 Activity Diagram

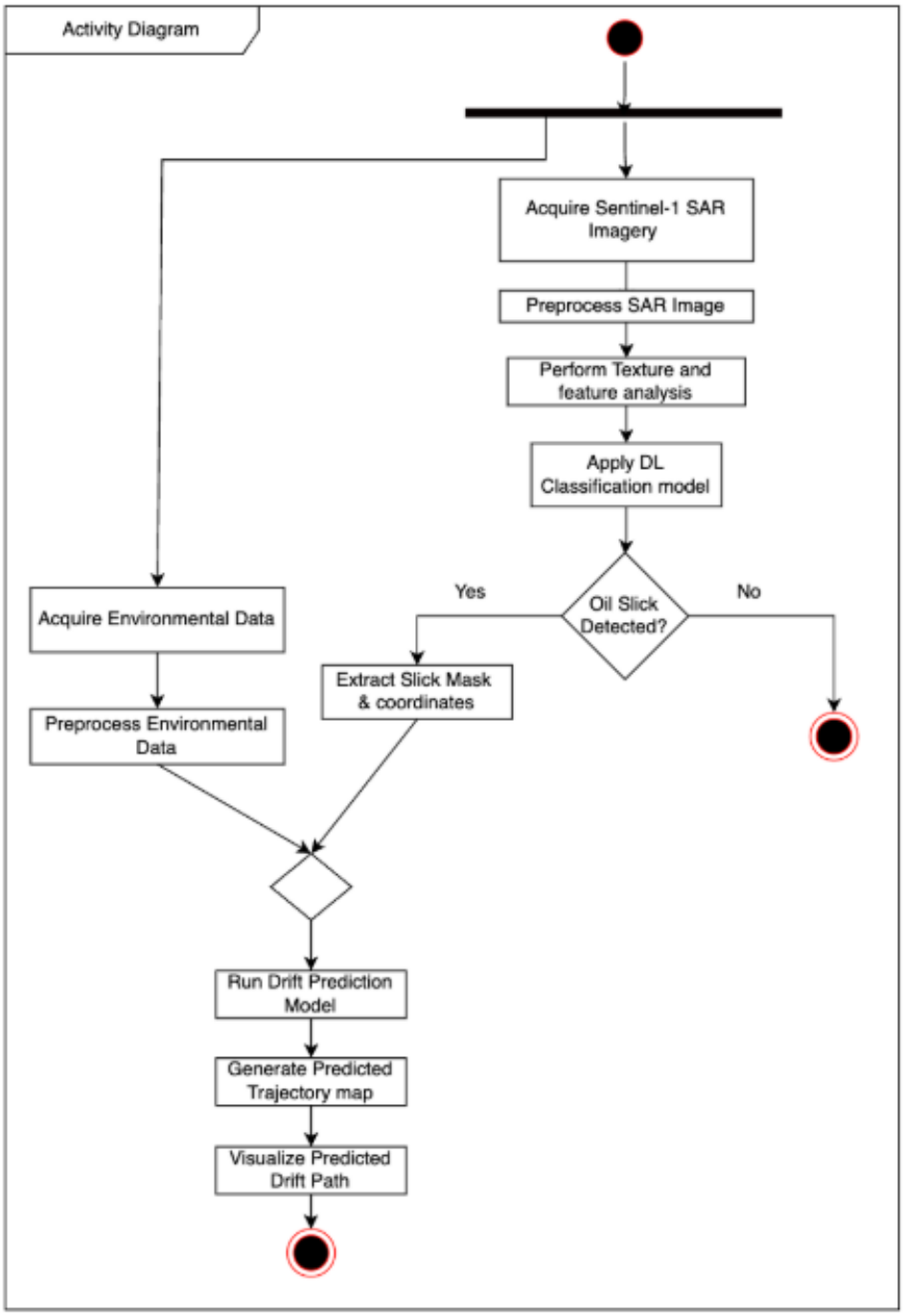


Figure 8.5: Activity Diagram

The activity diagram represents the complete workflow of the oil-slick detection and drift-prediction system, outlining how data flows from raw satellite imagery to the final predicted drift path. The process begins with the acquisition of Sentinel-1 SAR imagery, which serves as the primary input for the detection pipeline. Once the SAR

image is obtained, it is preprocessed to remove noise, normalize intensity values, and enhance overall image quality.

Following preprocessing, the system performs texture extraction and feature analysis to highlight characteristics that may indicate the presence of an oil slick. These features are then fed into a deep learning–based classification model, which determines whether an oil slick is present. If no slick is detected, the process terminates, and no further analysis is conducted. If a slick is detected, the system extracts the slick mask and its geographical coordinates for further processing.

Simultaneously, the workflow also gathers environmental data such as wind fields, ocean currents, and sea-surface conditions, which are essential for drift modelling. This environmental data undergoes preprocessing to ensure compatibility and consistency with the drift-prediction system. Once both the oil-slick features and environmental parameters are prepared, they are merged and passed into the drift-prediction model.

The drift-prediction model simulates the movement of the slick over time, considering dynamic environmental factors that influence its trajectory. The system then generates a predicted trajectory map showing how the slick is expected to move in the near future. Finally, this predicted drift path is visualized and presented as the output, enabling responders and analysts to make informed decisions for mitigation and cleanup planning.

8.2.4 Sequence Diagram

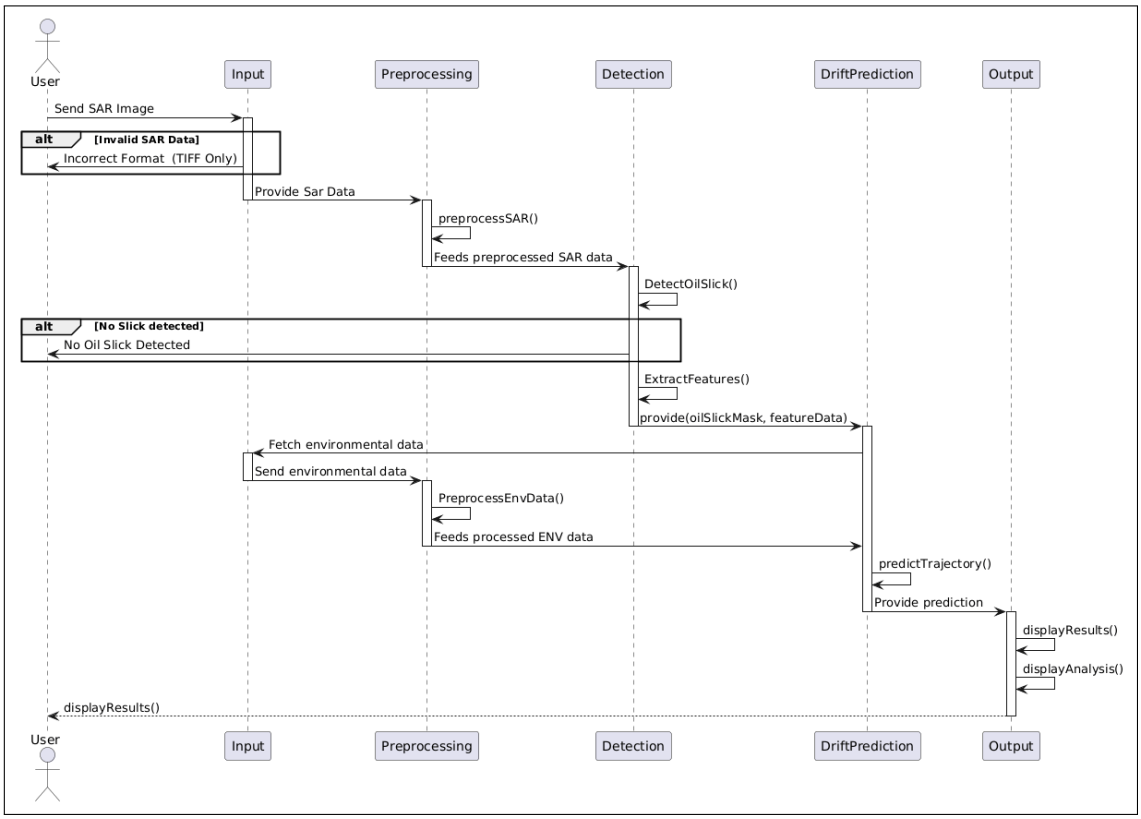


Figure 8.6: Sequence Diagram

The sequence diagram illustrates the complete workflow of an automated oil-slick detection and drift-prediction system built using SAR imagery. The interaction begins when the user provides a SAR image to the system through the Input module. The system first checks whether the uploaded file is in the correct TIFF format; if not, the user is notified about the invalid data and the process is terminated. Once the SAR data is validated, it is forwarded to the Preprocessing module.

In the preprocessing stage, the SAR image undergoes essential operations such as noise reduction, normalization, and resolution adjustment to prepare it for analysis. The preprocessed data is then sent to the Detection module, where deep-learning based algorithms attempt to identify the presence of oil slicks. If no oil slick is detected, the system directly informs the user. However, if a slick is detected, the system further extracts relevant features such as shape, intensity variation, texture, and boundary information.

These extracted features, along with the generated oil-slick mask, are passed to the DriftPrediction module. At this stage, the system fetches external environmental data, such as wind speed, ocean currents, and temperature, through the Input module. This environmental data is preprocessed and then supplied back to the drift-prediction model.

The drift-prediction model uses both the oil-slick features and environmental parameters to estimate the future movement or trajectory of the detected oil slick. A prediction output is generated and sent to the Output module.

Finally, the Output module presents the results to the user by displaying the detected slick, extracted features, predicted drift path, and relevant analytical visualizations. This ensures that users receive a complete and interpretable analysis, assisting authorities in planning response actions and understanding the progression of the oil spill.

CHAPTER 9
SYSTEM IMPLEMENTATION

9.1 TECHNOLOGY STACK

9.1.1 Programming Languages

– **Python** – Used for data preprocessing, SAR image analysis, deep learning model training, drift simulation, and backend workflow automation.

9.1.2 Frameworks and Libraries

Deep Learning

– **TensorFlow / Keras** – For building CNN, U-Net, and SegNet-based oil spill detection models.
– **PyTorch** – Used for advanced segmentation model development and experimentation.
– **Scikit-Learn** – For feature engineering and evaluation metrics.

Image Processing and Remote Sensing

– **OpenCV** – Noise reduction, filtering, and preprocessing of SAR images.
– **NumPy** – Numerical operations and matrix computations.
– **Rasterio** – Handling geospatial raster data from SAR satellites.

9.1.3 Satellite and Environmental Data Sources

– **Sentinel-1 SAR Dataset** – Primary dataset for oil slick detection.
– **OpenWeatherMap API** – Provides wind, current, temperature, and environmental parameters based on the latitude and longitude provided.

9.1.4 Drift Prediction Models and Tools

– **OpenDrift Framework** – An open-source Lagrangian particle tracking framework used to simulate the movement of oil slicks on the ocean surface.
‘‘‘

- **Particle-Based Simulation** – Represents the detected oil spill as a collection of virtual particles whose movement is influenced by environmental conditions.
- **Environmental Forcing Data** – Incorporates factors such as ocean currents, wind velocity and turbulent diffusion to predict spill movement.
- **Trajectory Prediction** – Forecasts the future position and spread of the oil slick over a predefined simulation period.
- **Visualization and Animation** – Generates drift trajectories and animations to visualize the predicted movement of the oil spill.

9.1.5 Development and Execution Environment

- **Operating System: Windows 11 , Unix** – Used for dataset preparation, model development, training, and drift prediction implementation.
- **GPU: NVIDIA GeForce RTX 4050 (6 GB)** – Utilized for training the ResNet classification model and U-Net segmentation model.
- **Development Environment: Jupyter Notebook and Visual Studio Code , SNAP** – Used for model development, experimentation, implementation, debugging, and visualization of results.

9.1.6 Visualization Tools

- **Matplotlib / Seaborn / Plotly** – Visualization of training results and forecast graphs.

9.1.7 Data Storage and File Formats

- **JSON** – Storage format for weather API data and configuration.
- **GeoTIFF** – For SAR images and segmentation mask outputs.
- **CSV** – For trajectory logs, numerical results, and dataset metadata.

9.2 MODULE SPLITUP

9.2.1 Input Module

Overview

The Input Module acts as the basic entry point of the whole system, and ensures that all necessary information is accurately collected, validate.The Input Module acts as the basic entry point of the whole system, and ensures that all necessary information is accurately collected, validated, and structured before any other process. Through the use of Synthetic Aperture Radar images that come from satellites with the help of specific APIs that collect the environmental parameters, the module helps achieve a systematic approach to oil spill detection and prediction.

Its ability to work with different data formats as well as conduct systematic validation helps ensure that the data that is processed afterwards is valid and accurate. Moreover, the exact positioning of satellite images in relation to the environmental factors provides consistency of all the analysis and predictions carried out in the further modules.

The structured and thoroughly validated data flow is beneficial not only for the process of detecting the oil spill and predicting its movement but also increases the effectiveness of the entire system. The Input Module is one of the key components of the pipeline that influences its performance and functionality since it is the point through which all the data flows.

Workflow

- Fetches SAR images of the Sentinel-1 satellite with high resolution through APIs like the Copernicus Open Access Hub or the datasets. The module guarantees the proper choice of acquisition parameters like polarization type (VV, VH), imaging type (IW/EW), and the time period to enable accurate monitoring.
- The auxiliary environmental parameters like wind speed/direction, sea surface temperature, ocean currents, and weather parameters are requested via Weather APIs or other oceanographic datasets. These parameters are crucial for accurately predicting the drift of oil spills.
- Supports raw file formats like GeoTIFF, or JSON.
- Once processed and structured, all input data will be passed to the Preprocessing Module.

9.2.2 Preprocessing Module

Overview

The Preprocessing Module is crucial in processing raw SAR images of the Sentinel-1 in order to detect and predict oil spills properly. The quality of the raw images obtained through Sentinel-1 SAR is highly susceptible to many problems, such as speckle noise, radiometric distortions, and geometric distortions which are caused by the basic nature of the microwave sensing technology. Differences in acquisition parameters, such as incidence angles, polarization modes, and environmental conditions, increase the difficulties of raw image acquisition.

In order to counter these issues, the role of the Preprocessing Module is to improve, normalize, and convert raw input data into a well-structured format that can be easily consumed by deep learning algorithms. The purpose of each of these steps is to clean the data, remove noise, keep spatial information relevant, and make sure everything is normalized consistently within the whole dataset. This preparation of input data is crucial in order to make sure the detection and prediction modules have the data of enough quality and consistency.

Workflow

- It takes in raw SAR image data that come from the user, Sentinel-1 API, or datasets.
- Then, it uses denoising filters like Lee filter, Frost filter, or median filter to remove speckle noise without distorting edges and structures. This is essential as the presence of noise might create oil slick appearance and cause false positive results.
- In SAR backscatter coefficient notation, there is (σ^0), which refers to the normalized radar cross-section.
- Terrain correction and georeferencing processes align SAR images to their actual geographic coordinates. This removes distortions from sensors' geometries and curvature of Earth.
- Images are resized to 512×512 resolution. This guarantees that all inputs to deep learning architectures (CNN and U-Net) have consistent size.
- Transformations like flipping, rotations, and scalings are applied to enhance variety in the dataset.

9.2.3 Detection Module

Overview

The function of the Detection Module is to detect and segment regions containing oil slicks from the processed SAR images by applying sophisticated deep learning techniques. SAR images typically possess a pattern where oil slicks are identified through dark areas that resemble look-alikes, including low-wind areas, biological films, and algal blooms. Hence, an efficient detection model will have to be able to learn spatial and contextual cues.

Deep learning models applied include Convolutional Neural Networks for classification and encoder-decoder models, such as U-Net and SegNet for segmentation of pixels. The CNNs can learn about the intensity, textures, and structures present to efficiently segment oil slicks from water regions and false positives.

Workflow

- Receives improved and standard SAR image from the Preprocessing Module, which ensures minimized noise and standardized format.
- Deploys convolutional networks for capturing spatial, textual, and context-based features such as intensity gradients, edges, and surface roughness features.
- Performs classification on region being oil slick or non-oil slick.
- Uses deep learning architecture such as U-Net to achieve pixel-level segmentation by creating highly accurate masks of oil spills.
- Generates binary or multi-class segmentation mask showing the regions where oil slick has been detected.

9.2.4 Drift Prediction Module

Overview

The Drift Prediction Module is responsible for forecasting the future movement of an oil slick after it has been detected and segmented from a Sentinel-1 SAR image. The module uses the output generated by the U-Net segmentation model to determine the location and extent of the oil spill. Geographical coordinates are extracted from the GeoTIFF metadata and used as the initial position for drift simulation.

The module employs the OpenDrift framework, which follows a Lagrangian particle-tracking approach. The detected oil slick is represented using virtual particles that move under the influence of environmental conditions such as ocean currents, wind, and turbulent diffusion. The simulation predicts the future trajectory and spread of the oil slick over a specified forecasting period and generates visualizations to support monitoring and response planning.

Workflow

- Receives the segmented oil slick mask generated by the Detection and Segmentation Module.
- Extracts geographical coordinates of the detected oil slick from the SAR GeoTIFF image using Rasterio.
- Initializes virtual particles representing the detected oil spill within the OpenDrift simulation environment.
- Incorporates environmental forcing data such as ocean currents and wind conditions to drive particle movement.
- Simulates the transport and dispersion of oil particles using the OpenDrift Lagrangian particle-tracking model.
- Predicts the future trajectory and spread of the oil slick over the simulation period.
- Generates drift animations and trajectory visualizations showing the expected movement of the oil spill.

9.2.5 Output and Visualization Module

Overview

The Output and Visualization Module is tasked with the role of compiling and translating the output generated by the system into an easily digestible form. This module is the interface between the outputs of models that may be difficult to interpret and end users such as researchers and decision makers.

The Output and Visualization Module incorporates output from the Detection and Drift Prediction modules, which include segmentation of regions affected by oil spills and the prediction of their drift path. Through geospatial visualization, this module translates this data into easily interpretable maps and projections.

Workflow

- Inputs include the segmented oil slick mask generated by the Detection Module and the trajectory predictions generated by the Drift Prediction Module, along with associated spatial and temporal metadata.
- The detection outputs and drift prediction are aligned spatially via georeferencing to ensure consistency in the visual representation.
- A variety of data layers are combined to generate visual output, such as the base SAR image and detection results.
- Visual outputs are displayed for interpretation.
- Outputs can be exported in several formats, including PNG, JSON and PDF.

9.3 ACTUAL CODE

9.3.1 Data Loading

```
import rasterio
import numpy as np
import tensorflow as tf
import matplotlib.pyplot as plt

def load_sar_tiff(path):
    def _read(p):
        path_str = p.item()

        with rasterio.open(path_str) as src:
            vv = src.read(1).astype(np.float32)
            vh = src.read(2).astype(np.float32)

        # Fixed-range normalization (dB)
        vv = np.clip(vv, -35.0, 5.0)
        vh = np.clip(vh, -40.0, 0.0)

        vv = (vv + 35.0) / 40.0
        vh = (vh + 40.0) / 40.0

        img = np.stack([vv, vh], axis=-1)

        # Resize 2048 -> 512
        img = tf.image.resize(img, (512, 512), method="bilinear
            ").numpy()

        return img.astype(np.float32)

    img = tf.numpy_function(_read, [path], tf.float32)
    img.set_shape([512, 512, 2])
    return img
```

9.3.2 Data Preprocessing

```python
import rasterio
import numpy as np
import cv2
from scipy.ndimage import uniform_filter

# Convert to dB
def to_db(img):
    img = np.clip(img, 1e-6, None)
    return 10 * np.log10(img)

# Lee Filter
def lee_filter(img, size=5):
    mean = uniform_filter(img, size)
    mean_sq = uniform_filter(img**2, size)
    variance = mean_sq - mean**2

    overall_var = np.var(img)

    weights = variance / (variance + overall_var + 1e-8)
    filtered = mean + weights * (img - mean)

    return filtered

# Normalization
def normalize(img):
    return (img - np.mean(img)) / (np.std(img) + 1e-8)
```

9.3.3 Detection (CNN) Model Training

```
import os
import random
import numpy as np
import tensorflow as tf
import rasterio

from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv2D, MaxPooling2D,
   Flatten, Dense
from tensorflow.keras.callbacks import EarlyStopping

# Config
IMG_SIZE = (512, 512)
BATCH_SIZE = 8
EPOCHS = 5
SEED = 42
os.environ["PYTHONHASHSEED"] = str(SEED)
random.seed(SEED)
np.random.seed(SEED)
tf.random.set_seed(SEED)

# Dataset Base Path
BASE_PATH = "PATH"

TRAIN_IMG_DIR = os.path.join(BASE_PATH, "Train", "Images")
TEST_IMG_DIR  = os.path.join(BASE_PATH, "Test", "Images")

# Image Loading
def load_sar_tiff(path):
    def _read(p):
        with rasterio.open(p.decode()) as src:
            vv = src.read(1).astype(np.float32)
            vh = src.read(2).astype(np.float32)

        vv = np.clip(vv, -35, 5)
        vh = np.clip(vh, -40, 0)

        vv = (vv + 35) / 40
        vh = (vh + 40) / 40

        img = np.stack([vv, vh], axis=-1)
        img = tf.image.resize(img, IMG_SIZE).numpy()
        return img

    img = tf.numpy_function(_read, [path], tf.float32)
    img.set_shape([512, 512, 2])
    return img

# Image Augmentation
def augment(image, label):
    image = tf.image.random_flip_left_right(image)
```

```
    image = tf.image.random_flip_up_down(image)

    k = tf.random.uniform([], 0, 4, dtype=tf.int32)
    image = tf.image.rot90(image, k)

    tx = tf.random.uniform([], -0.05, 0.05)
    ty = tf.random.uniform([], -0.05, 0.05)

    image = tf.roll(
        image,
        shift=[
            tf.cast(tx * IMG_SIZE[0], tf.int32),
            tf.cast(ty * IMG_SIZE[1], tf.int32)
        ],
        axis=[0, 1]
    )

    return image, label

# Balanced Dataset Building
def build_balanced_dataset(images_root, target_per_class=1200):
    oil_dir = os.path.join(images_root, "Oil")
    no_oil_dir = os.path.join(images_root, "No_Oil")
    lookalike_dir = os.path.join(images_root, "Lookalike")

    oil_files = sorted([os.path.join(oil_dir, f) for f in os.
       listdir(oil_dir) if f.endswith(".tif")])
    no_oil_files = sorted([os.path.join(no_oil_dir, f) for f in
        os.listdir(no_oil_dir) if f.endswith(".tif")])
    lookalike_files = sorted([os.path.join(lookalike_dir, f)
       for f in os.listdir(lookalike_dir) if f.endswith(".tif")
       ])

    combined_no_oil = no_oil_files + lookalike_files
    random.shuffle(combined_no_oil)

    oil_files = oil_files[:target_per_class]
    combined_no_oil = combined_no_oil[:target_per_class]

    paths = oil_files + combined_no_oil
    labels = [1]*len(oil_files) + [0]*len(combined_no_oil)

    return np.array(paths), np.array(labels)

# Test Dataset Building
def build_test_dataset(images_root):
    oil_dir = os.path.join(images_root, "Oil")
    no_oil_dir = os.path.join(images_root, "No_Oil")
    lookalike_dir = os.path.join(images_root, "Lookalike")

    oil_files = [os.path.join(oil_dir, f) for f in os.listdir(
       oil_dir) if f.endswith(".tif")]
```

```
    no_oil_files = [os.path.join(no_oil_dir, f) for f in os.listdir(no_oil_dir) if f.endswith(".tif")]
    lookalike_files = [os.path.join(lookalike_dir, f) for f in os.listdir(lookalike_dir) if f.endswith(".tif")]

    paths = oil_files + no_oil_files + lookalike_files
    labels = (
        [1] * len(oil_files) +
        [0] * len(no_oil_files) +
        [0] * len(lookalike_files)
    )

    return np.array(paths), np.array(labels)

from sklearn.model_selection import train_test_split

all_paths, all_labels = build_balanced_dataset(TRAIN_IMG_DIR)


# Printing Total Images
train_paths, val_paths, train_labels, val_labels = train_test_split(
    all_paths,
    all_labels,
    train_size=0.66,
    stratify=all_labels,
    random_state=SEED
)

print("Train samples:", len(train_paths))
print("Val samples:", len(val_paths))

# Dataset Builder Function
def make_dataset(paths, labels, augment_fn=None, shuffle=False):
    ds = tf.data.Dataset.from_tensor_slices((paths, labels))

    if shuffle:
        ds = ds.shuffle(
            buffer_size=len(paths),
            seed=SEED,
            reshuffle_each_iteration=True
        )

    ds = ds.map(
        lambda x, y: (load_sar_tiff(x), y),
        num_parallel_calls=1  # rasterio-safe
    )

    if augment_fn is not None:
        ds = ds.map(augment_fn, num_parallel_calls=1)

    ds = ds.batch(BATCH_SIZE)
```

```
    ds = ds.prefetch(tf.data.AUTOTUNE)

    return ds

train_ds = make_dataset(
    train_paths,
    train_labels,
    augment_fn=augment,
    shuffle=True
)

val_ds = make_dataset(
    val_paths,
    val_labels,
    augment_fn=None,
    shuffle=False
)

test_ds = make_dataset(
    test_paths,
    test_labels,
    augment_fn=None,
    shuffle=False
)

# Model Architecture
def build_cnn():
    model = Sequential()

    for i in range(6):
        if i == 0:
            model.add(Conv2D(
                filters=32,
                kernel_size=(3, 3),
                activation="relu",
                padding="same",
                input_shape=(512, 512, 2)
            ))
        else:
            model.add(Conv2D(
                filters=32,
                kernel_size=(3, 3),
                activation="relu",
                padding="same"
            ))

        model.add(MaxPooling2D(pool_size=(2, 2)))


    model.add(Flatten())

    model.add(Dense(20, activation="relu"))
    model.add(Dense(20, activation="relu"))
```

```
    model.add(Dense(1, activation="sigmoid"))


    model.compile(
        optimizer=tf.keras.optimizers.Adam(),
        loss="binary_crossentropy",
        metrics=["accuracy"]
    )

    return model

# Training Constraints
early_stop = EarlyStopping(
    monitor="val_loss",
    patience=8,
    restore_best_weights=True
)

checkpoint_cb = tf.keras.callbacks.ModelCheckpoint(
    filepath="CNN_checkpoints/model_epoch_{epoch}.weights.h5",
    save_weights_only=True,
    save_best_only=False
)

model = build_cnn()
history = model.fit(
    train_ds,
    validation_data=val_ds,
    epochs=50,
    callbacks=[
        early_stop,
        checkpoint_cb
    ]
)
```

9.3.4 Training and Validation Result Visualization

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.metrics import confusion_matrix,
   classification_report

y_true = []
y_pred = []

for x_batch, y_batch in test_ds:
    preds = model.predict(x_batch, verbose=0)
    preds = (preds > 0.35).astype(int)   # threshold = 0.5

    y_true.extend(y_batch.numpy())
    y_pred.extend(preds.flatten())

y_true = np.array(y_true)
y_pred = np.array(y_pred)

cm = confusion_matrix(y_true, y_pred)
print("Confusion Matrix:")
print(cm)

test_loss, test_acc = model.evaluate(test_ds)
print(f"Test Loss: {test_loss:.4f}")
print(f"Test Accuracy: {test_acc:.4f}")

val_loss, val_acc = model.evaluate(val_ds)
print(f"Val Loss: {val_loss:.4f}")
print(f"Val Accuracy: {val_acc:.4f}")
```

9.3.5 Segmentation (U-Net) Model Training

```
import os
import json
import random
import glob
import numpy as np
import matplotlib.pyplot as plt
import tensorflow as tf
import rasterio

from tensorflow.keras import backend as K
from tensorflow.keras.models import Model
from tensorflow.keras.layers import (
    Input, Conv2D, MaxPooling2D, UpSampling2D,
    concatenate, BatchNormalization, Activation, Dropout
)
from tensorflow.keras.callbacks import ReduceLROnPlateau
from tensorflow.keras.optimizers import Adam
from sklearn.model_selection import train_test_split

# Config
SEED = 42
os.environ['PYTHONHASHSEED'] = str(SEED)
random.seed(SEED)
np.random.seed(SEED)
tf.random.set_seed(SEED)
BASE_PATH = r"C:\FinalYear\Dataset-OG"

TRAIN_IMG_DIR  = os.path.join(BASE_PATH, 'Train', 'Images')
TRAIN_MASK_DIR = os.path.join(BASE_PATH, 'Train', 'Masks')
TEST_IMG_DIR   = os.path.join(BASE_PATH, 'Test',  'Images')
TEST_MASK_DIR  = os.path.join(BASE_PATH, 'Test',  'Masks')

CKPT_DIR     = "unet_checkpoints"   # folder for all per-epoch .h5 files
HISTORY_FILE = "unet_full_history.json"  # accumulates metrics across all runs
os.makedirs(CKPT_DIR, exist_ok=True)

# Hyperparameters
IMG_SIZE = (512, 512)
BATCH_SIZE = 4
EPOCHS_PER_RUN = 10
LR = 1e-4

# Focal Loss parameters
FOCAL_ALPHA = 0.25   # weight for the positive (oil) class
FOCAL_GAMMA = 2.0    # focusing parameter; 0 = standard cross-entropy

# U-Net filter schedule
FILTERS = [16, 32, 64, 128, 256, 512, 1024]
```

```
49
50
51  # Custom Metrics and Loss
52  # IoU metric
53  def iou_metric(y_true, y_pred, smooth=1e-6):
54      y_pred = tf.cast(y_pred > 0.5, tf.float32)
55      y_true = tf.cast(y_true, tf.float32)
56      intersection = K.sum(y_true * y_pred, axis=[1, 2, 3])
57      union = K.sum(y_true, axis=[1, 2, 3]) + K.sum(y_pred, axis
            =[1, 2, 3]) - intersection
58      return K.mean((intersection + smooth) / (union + smooth))
59
60
61  # Focal Loss
62  def focal_loss(alpha=FOCAL_ALPHA, gamma=FOCAL_GAMMA):
63      def loss_fn(y_true, y_pred):
64          y_true = tf.cast(y_true, tf.float32)
65          y_pred = tf.clip_by_value(y_pred, 1e-7, 1.0 - 1e-7)
66          bce_pos = -y_true * tf.math.log(y_pred)
67          bce_neg = -(1.0 - y_true) * tf.math.log(1.0 - y_pred)
68          focal_pos = alpha       * tf.pow(1.0 - y_pred, gamma) *
                bce_pos
69          focal_neg = (1 - alpha) * tf.pow(y_pred, gamma)       *
                bce_neg
70          return tf.reduce_mean(focal_pos + focal_neg)
71      return loss_fn
72
73
74  # Dice coefficient (supplementary metric)
75  def dice_coef(y_true, y_pred, smooth=1e-6):
76      y_pred = tf.cast(y_pred > 0.5, tf.float32)
77      y_true = tf.cast(y_true, tf.float32)
78      intersection = K.sum(y_true * y_pred, axis=[1, 2, 3])
79      return K.mean((2.0 * intersection + smooth) /
80                    (K.sum(y_true, axis=[1, 2, 3]) + K.sum(y_pred
                         , axis=[1, 2, 3]) + smooth))
81
82
83  # U-Net Architecture
84  def conv_block(x, filters, dropout_rate=0.0):
85      x = Conv2D(filters, (3, 3), padding='same',
            kernel_initializer='he_normal')(x)
86      x = BatchNormalization()(x)
87      x = Activation('relu')(x)
88      x = Conv2D(filters, (3, 3), padding='same',
            kernel_initializer='he_normal')(x)
89      x = BatchNormalization()(x)
90      x = Activation('relu')(x)
91      if dropout_rate > 0:
92          x = Dropout(dropout_rate)(x)
93      return x
94
95
```

```
def build_unet(input_shape=(512, 512, 2), filters=None):
    if filters is None:
        filters = FILTERS   # [16, 32, 64, 128, 256, 512, 1024]

    inputs = Input(shape=input_shape, name='sar_input')

    # Encoder
    skips = []
    x = inputs
    for i, f in enumerate(filters[:-1]):
        dr = 0.1 if i >= len(filters) - 3 else 0
        x = conv_block(x, f, dropout_rate=dr)
        skips.append(x)
        x = MaxPooling2D((2, 2))(x)

    # Bottleneck
    x = conv_block(x, filters[-1], dropout_rate=0.2)

    # Decoder
    for f, skip in zip(reversed(filters[:-1]), reversed(skips)):
        x = UpSampling2D((2, 2))(x)
        x = concatenate([x, skip])
        x = conv_block(x, f)

    # Output: 1x1 conv -> sigmoid
    outputs = Conv2D(1, (1, 1), activation='sigmoid', name='segmentation_map')(x)

    model = Model(inputs=inputs, outputs=outputs, name='Depth_UNet')
    return model

# Data Loading
def load_sar_image(path):
    with rasterio.open(path) as src:
        vv = src.read(1).astype(np.float32)
        vh = src.read(2).astype(np.float32)
    vv = np.clip(vv, -35, 5)
    vh = np.clip(vh, -40, 0)
    vv = (vv + 35) / 40.0
    vh = (vh + 40) / 40.0
    img = np.stack([vv, vh], axis=-1)
    img = tf.image.resize(img, IMG_SIZE).numpy()
    return img


def load_mask(path):
    with rasterio.open(path) as src:
        mask = src.read(1).astype(np.float32)
    mask = (mask > 0).astype(np.float32)
    mask = tf.image.resize(mask[..., np.newaxis], IMG_SIZE, method='nearest').numpy()
```

```
    return mask


def build_segmentation_dataset(images_root, masks_root):
    oil_img_dir  = os.path.join(images_root, 'Oil')
    oil_mask_dir = os.path.join(masks_root,  'Oil')
    img_files  = sorted(glob.glob(os.path.join(oil_img_dir,  '*.tif')))
    mask_files = sorted(glob.glob(os.path.join(oil_mask_dir, '*.tif')))
    assert len(img_files) == len(mask_files)
    return np.array(img_files), np.array(mask_files)


def data_generator(img_paths, mask_paths, batch_size, augment=False, shuffle=True):
    n = len(img_paths)
    indices = np.arange(n)
    while True:
        if shuffle:
            np.random.shuffle(indices)
        for start in range(0, n, batch_size):
            batch_idx = indices[start:start + batch_size]
            imgs, masks = [], []
            for i in batch_idx:
                img  = load_sar_image(img_paths[i])
                mask = load_mask(mask_paths[i])
                if augment:
                    if random.random() > 0.5:
                        img  = np.fliplr(img);  mask = np.fliplr(mask)
                    if random.random() > 0.5:
                        img  = np.flipud(img);  mask = np.flipud(mask)
                    k = random.choice([0, 1, 2, 3])
                    img  = np.rot90(img,  k)
                    mask = np.rot90(mask, k)
                imgs.append(img)
                masks.append(mask)
            yield np.array(imgs, dtype=np.float32), np.array(masks, dtype=np.float32)


all_img_paths, all_mask_paths = build_segmentation_dataset(TRAIN_IMG_DIR, TRAIN_MASK_DIR)

# 66% train / 34% validation
train_imgs, val_imgs, train_masks, val_masks = train_test_split(
    all_img_paths, all_mask_paths, test_size=0.34, random_state=SEED
)

```

```
train_steps = max(1, len(train_imgs) // BATCH_SIZE)
val_steps   = max(1, len(val_imgs)   // BATCH_SIZE)

train_gen = data_generator(train_imgs, train_masks, BATCH_SIZE,
    augment=True,  shuffle=True)
val_gen   = data_generator(val_imgs,   val_masks,   BATCH_SIZE,
    augment=False, shuffle=False)

# Callbacks
class EpochCheckpoint(tf.keras.callbacks.Callback):
    def __init__(self, ckpt_dir, epoch_offset=0):
        super().__init__()
        self.ckpt_dir     = ckpt_dir
        self.epoch_offset = epoch_offset

    def on_epoch_end(self, epoch, logs=None):
        global_epoch = self.epoch_offset + epoch + 1   # 1-
           based
        path = os.path.join(self.ckpt_dir, f'unet_epoch_{
           global_epoch:03d}.h5')
        self.model.save(path)
        iou = logs.get('val_iou_metric', float('nan'))


reduce_lr = ReduceLROnPlateau(
    monitor='val_iou_metric', mode='max',
    factor=0.5, patience=5, min_lr=1e-7, verbose=1
)

unet_model = build_unet()
unet_model.compile(
    optimizer=Adam(learning_rate=LR),
    loss=focal_loss(alpha=FOCAL_ALPHA, gamma=FOCAL_GAMMA),
    metrics=[iou_metric, dice_coef, 'accuracy']
)

run_callbacks = [
    EpochCheckpoint(CKPT_DIR, epoch_offset=epoch_offset),
    reduce_lr,
]

history_run = unet_model.fit(
    train_gen,
    steps_per_epoch=train_steps,
    epochs=EPOCHS_PER_RUN,
    validation_data=val_gen,
    validation_steps=val_steps,
    callbacks=run_callbacks,
    verbose=1
)
```

9.3.6 Drift Prediction

```
from opendrift.models.oceandrift import OceanDrift
from datetime import datetime, timedelta
import numpy as np

# Simulation Configuration
SIMULATION_DURATION_HOURS = 48
NUM_PARTICLES = 1000
TIME_STEP = 900

# Oil Spill Coordinate Extraction
spill_pixels = extract_spill_coordinates(
    file_bytes,
    oil_mask
)

# OpenDrift Environment Setup
drift_model = setup_drift_environment(
    spill_pixels,
    start_date,
    end_date
)

# Particle Sampling
if len(spill_pixels) > NUM_PARTICLES:

    indices = np.random.choice(
        len(spill_pixels),
        NUM_PARTICLES,
        replace=False
    )

    sampled_pixels = spill_pixels[indices]

else:
    sampled_pixels = spill_pixels

# Latitude and Longitude Extraction
lons = sampled_pixels[:, 0]
lats = sampled_pixels[:, 1]

# Particle Initialization
drift_model.seed_elements(
    lon=lons,
    lat=lats,
    time=datetime.strptime(
        start_date,
        "%Y-%m-%d"
    )
)

# Drift Simulation Execution
```

```
drift_model.run(
    duration=timedelta(
        hours=SIMULATION_DURATION_HOURS
    ),
    time_step=TIME_STEP
)

# Drift Statistics Calculation
lon_start = drift_model.result.lon.values[0]
lat_start = drift_model.result.lat.values[0]

lon_end = drift_model.result.lon.values[-1]
lat_end = drift_model.result.lat.values[-1]

center_start = (
    np.mean(lon_start),
    np.mean(lat_start)
)

center_end = (
    np.mean(lon_end),
    np.mean(lat_end)
)

# Drift Map Generation
drift_map = create_drift_map_html(
    center_start,
    center_end
)

# Animation Generation
generate_drift_animation(
    drift_model,
    "oil_drift_prediction.mp4"
)
```

CHAPTER 10

TEST CASES

10.1 INPUT VERIFICATION

Test cases were created to verify that the system accepts only valid Synthetic Aperture Radar (SAR) image formats for processing. The proposed model is designed to accept satellite images in TIFF (.tif) format. Any other image format cannot be uploaded. This ensures that only properly formatted SAR images are provided to the segmentation pipeline, thereby maintaining consistency and preventing errors during oil spill detection

10.2 DATASET TEST CASES

Different SAR image samples from the dataset were used to evaluate the performance of the proposed U-Net model. Each test case includes the original SAR image, the corresponding ground truth mask, and the mask generated by the model. Performance metrics such as prediction class, accuracy, Intersection over Union (IoU) score, spill area, and confidence value were recorded to analyze the effectiveness of the oil spill segmentation process.

The generated masks were compared with the original masks to verify the ability of the model to correctly identify oil-contaminated regions. Experimental results demonstrate that the proposed system can successfully distinguish between oil spill and non-oil regions with satisfactory accuracy and segmentation performance.

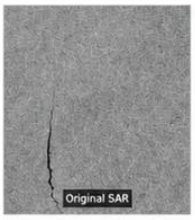
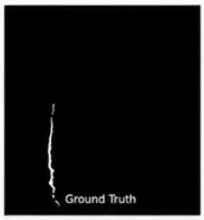
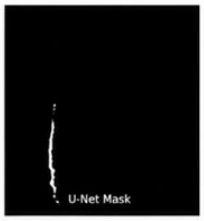
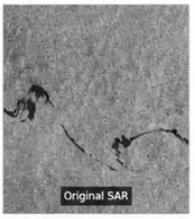
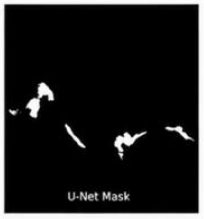
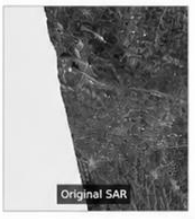
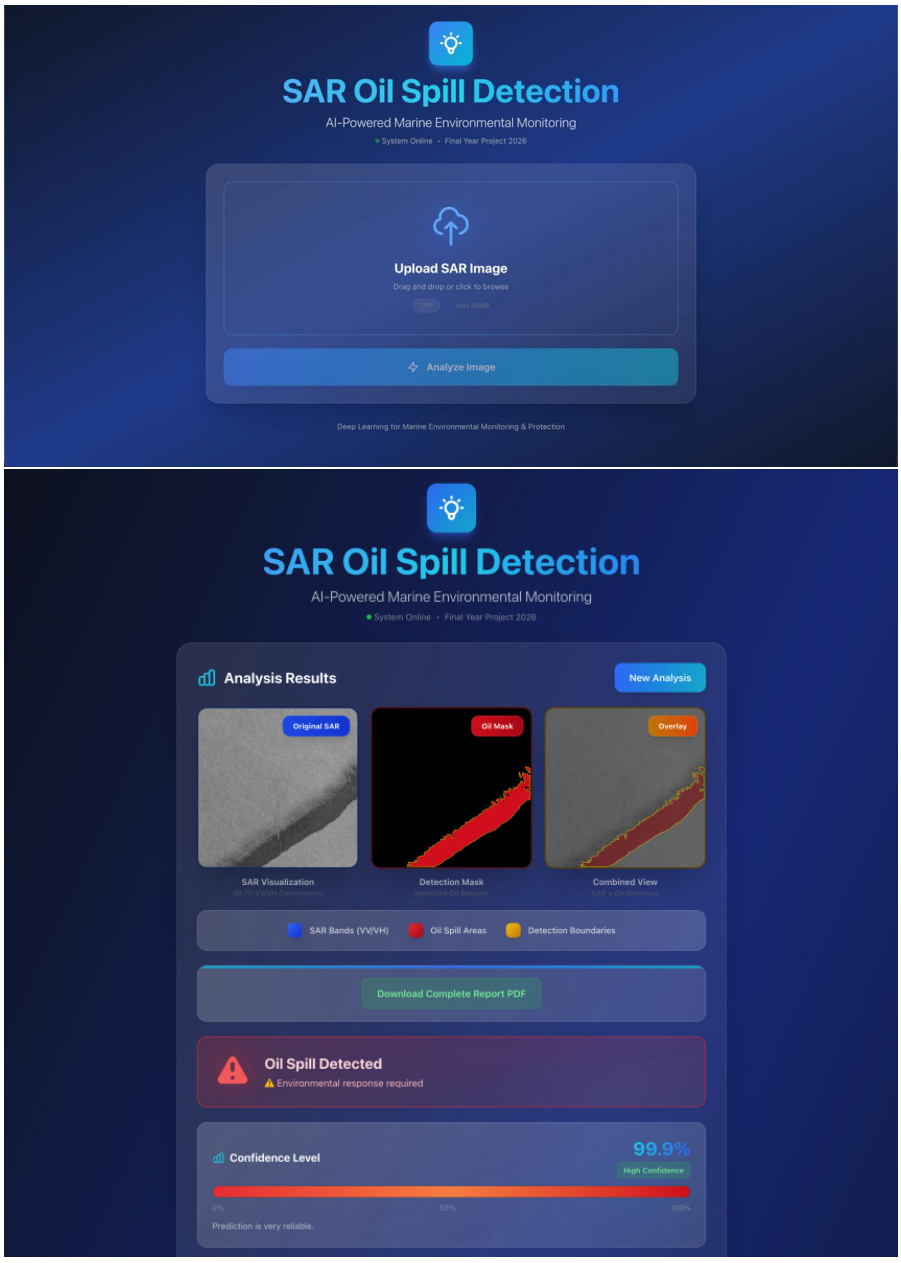
S. No.	Sample Data ID	Original SAR Image	Validation Metric Results	Original Mask	Generated Mask
1.	00062.tif		Metric: Value Pred Class: Oil True Class: Oil Accuracy: 0.9985 IoU Score: 0.7267 Spill Area: 0.13 km² Confidence: 61.34%		
2.	00006.tif		Metric: Value Pred Class: No_Oil True Class: Oil Accuracy: 0.7185 IoU Score: 0.3902 Spill Area: 4.97 km² Confidence: 93.54%		
3.	00070.tif		Metric: Value Pred Class: No_Oil True Class: Oil Accuracy: 0.9819 IoU Score: 0.5307 Spill Area: 0.57 km² Confidence: 90.88%		
4.	00057.tif		Metric: Value Pred Class: No_Oil True Class: No_Oil Accuracy: 1.0000 IoU Score: 0.0000 Spill Area: 0.00 km² Confidence: 100.00%		
5.	00028.tif		Metric: Value Pred Class: Oil True Class: Oil Accuracy: 0.9733 IoU Score: 0.5047 Spill Area: 0.75 km² Confidence: 76.08%		
6.	00035.tif		Metric: Value Pred Class: No_Oil True Class: No_Oil Accuracy: 1.0000 IoU Score: 0.0000 Spill Area: 0.00 km² Confidence: 92.22%		

Figure 10.1: Test Cases

CHAPTER 11

GUI AND RESULTS

11.1 GUI



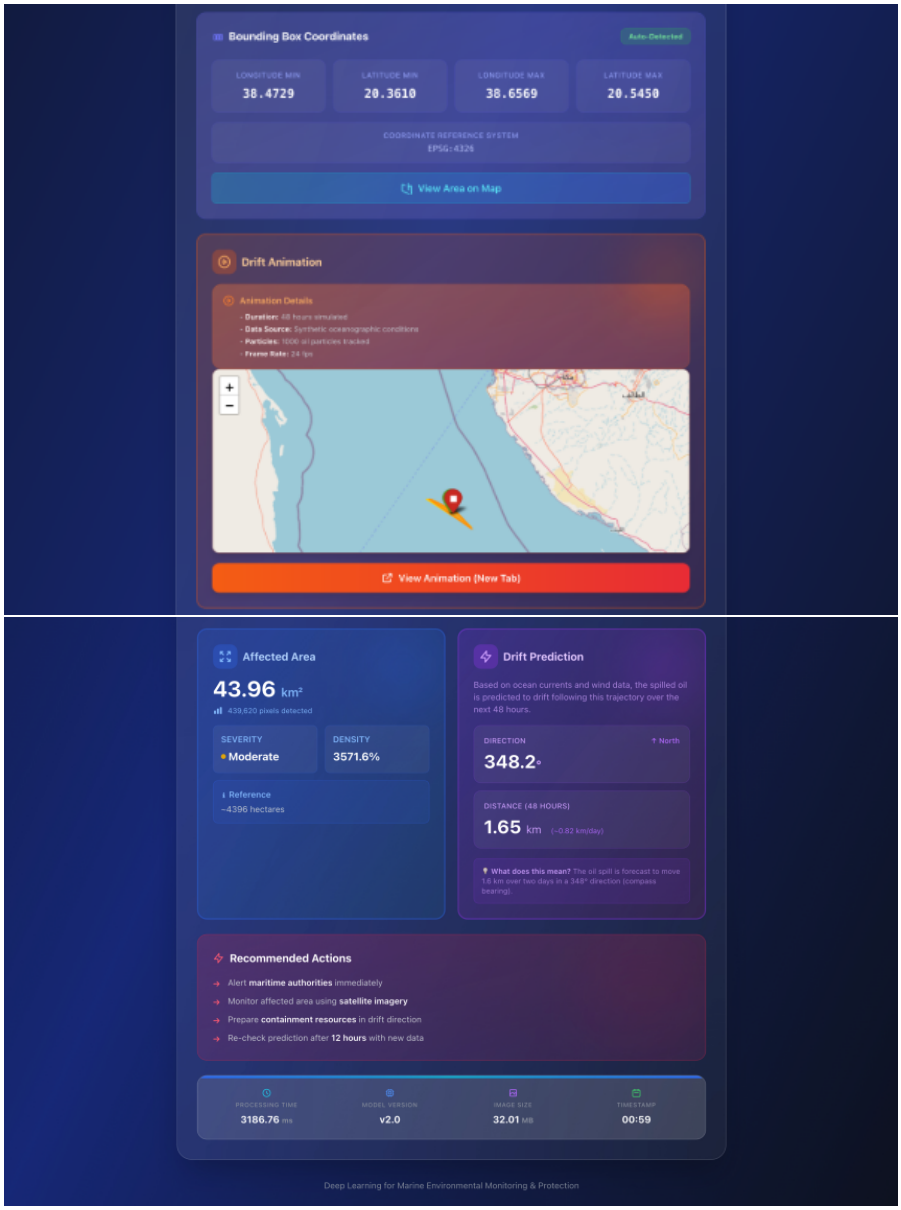


Figure 11.1: GUI Screenshots

The graphical user interface (GUI) for the oil slick detection application is developed using React for the frontend and Flask for the backend. The web application provides a simple and interactive platform for uploading SAR images and visualizing the segmentation results generated by the deep learning model. The React-Flask architecture ensures smooth communication between the user interface and the U-Net model, enabling efficient prediction and visualization of oil spill regions.

11.1.1 Key Features:

• Drag-and-Drop File Upload:

The interface provides a drag-and-drop upload mechanism that allows users to easily upload SAR images for analysis. This feature improves usability and simplifies the image submission process.

• SAR Image Upload Validation: The system accepts SAR images in supported formats and validates the uploaded files before processing. This ensures that only valid input images are passed to the segmentation model.

• Real-Time Oil Spill Detection:

After image upload, the Flask backend preprocesses the image and performs inference using the trained U-Net model. The prediction results are generated automatically and displayed to the user.

• Oil Spill Mask Visualization:

The application displays the generated segmentation mask corresponding to the uploaded SAR image, allowing users to visually identify oil-contaminated regions.

• Spill Area Calculation:

The system estimates the area occupied by the detected oil spill region. This information assists in understanding the extent of contamination and supports environmental assessment.

• Drift Prediction for 24–48 Hours:

The proposed system predicts the future movement of the detected oil spill and estimates its drift trajectory for the next 24 to 48 hours. This helps in monitoring the spread of oil slicks and supports timely response planning.

• Performance Metrics Display:

Important parameters such as prediction confidence, IoU score, accuracy, and spill area are presented to provide quantitative evaluation of model performance.

• Web-Based Accessibility:

The application can be accessed through modern web browsers without requiring additional software installation, making the system convenient and easy to use.

Advantages of Using React and Flask for the GUI

• Modular Architecture:

The separation of frontend and backend components improves maintainability and simplifies future development.

• Faster User Interaction:

React provides efficient rendering and smooth user experience through reusable components.

• Lightweight Backend Framework:

Flask offers a flexible framework for handling API requests and integrating deep learning models.

• Seamless Integration with Machine Learning Models:

The backend supports efficient execution of the U-Net model and enables real-time prediction.

• Scalability and Extensibility:

Additional functionalities and analytical modules can easily be incorporated into the application in future enhancements.

Table 11.1: GUI Capabilities

Feature	**Description**
Drag-and-Drop Upload	Easy image upload through drag-and-drop mechanism.
Oil Spill Detection	Automatic segmentation of oil spill regions using the U-Net model.
Mask Visualization	Displays the predicted oil spill mask generated by the model.
Spill Area Calculation	Computes the estimated area affected by the detected oil spill.
Drift Prediction	Predicts the movement of the oil spill for the next 24–48 hours.
Metric Display	Shows evaluation parameters such as accuracy, IoU score, and confidence.
Backend Processing	Flask API integrated with the deep learning model for efficient processing.
Accessibility	Browser-based application that does not require additional software installation.

11.2 EXPERIMENTAL RESULTS

In the field of remote sensing and marine environmental monitoring, numerous image segmentation techniques have been proposed for oil spill detection. Based on literature survey and experimental observations, the U-Net architecture was selected for segmenting oil-contaminated regions from SAR imagery. The model was trained using SAR images and their corresponding masks to accurately identify oil spill regions. In addition to segmentation, the proposed system estimates spill area and predicts the drift trajectory for the next 24–48 hours. The obtained results demonstrate the effectiveness of the proposed framework for automated oil spill monitoring.

A. Dataset:

Oil Spill Detection Dataset:

The proposed system utilizes Synthetic Aperture Radar (SAR) images collected from publicly available oil spill datasets [25]. The dataset contains SAR images together with their corresponding ground truth masks representing oil-contaminated regions.

The images are stored in TIFF (.tif) format and include both oil spill and non-oil samples. The availability of manually annotated masks enables supervised learning and facilitates accurate segmentation of oil spill regions.

The dataset was divided into training, validation, and testing subsets to ensure proper model evaluation and improve generalization capability.

Table 11.2: Dataset Distribution

	Oil	**No_Oil + Lookalike**
Training	1200	650 + 650
Testing	150	150 + 150

B. Preprocessing:

Preprocessing is a crucial step in preparing Sentinel-1 SAR imagery for deep learning-based oil spill detection and segmentation. The acquired dual-polarization SAR images (VV and VH) were first processed using SNAP software, where orbit correction, thermal noise removal, radiometric calibration, speckle filtering, and terrain correction were applied to generate calibrated Sigma0 backscatter images in decibel (dB) format. The images were then divided into smaller patches and converted into a standardized format suitable for model training.

To improve model performance, the VV and VH channels were combined as a two-channel input, followed by intensity normalization and clipping to reduce the effect of extreme backscatter values. Ground truth masks were created for each image, with oil spill regions labeled as foreground and all other regions labeled as background. Finally, the dataset was split into training and validation sets, and data augmentation techniques such as rotation, translation, and flipping were applied to increase data diversity, reduce

overfitting, and improve the generalization capability of both the CNN and U-Net models.

C. Evaluation Metrics:

Intersection over Union (IoU)

IoU is used to evaluate the overlap between the predicted mask and the ground truth mask. Higher IoU values indicate better segmentation performance.

Accuracy

Accuracy measures the percentage of correctly classified pixels and provides an overall indication of model performance.

Confidence Score

The confidence score represents the probability associated with the predicted class and indicates the reliability of model predictions.

Spill Area Estimation

Spill area estimation calculates the approximate area occupied by the detected oil spill region. This information is useful for environmental assessment and emergency response planning.

D. Visual Results:

The proposed U-Net model successfully generated segmentation masks corresponding to oil spill regions present in SAR images. Visual comparison between the ground truth masks and the generated masks demonstrates that the model can accurately identify contaminated regions while minimizing background noise.

The predicted masks were further utilized to estimate the affected spill area and perform drift prediction analysis. Experimental observations indicate that the model performs effectively on both oil spill and non-oil samples. Images containing distinct spill patterns achieved higher IoU and accuracy values, while minor variations were observed for complex SAR textures.

Overall, the obtained results validate the effectiveness of the proposed approach for automatic oil slick detection and drift prediction using SAR imagery.

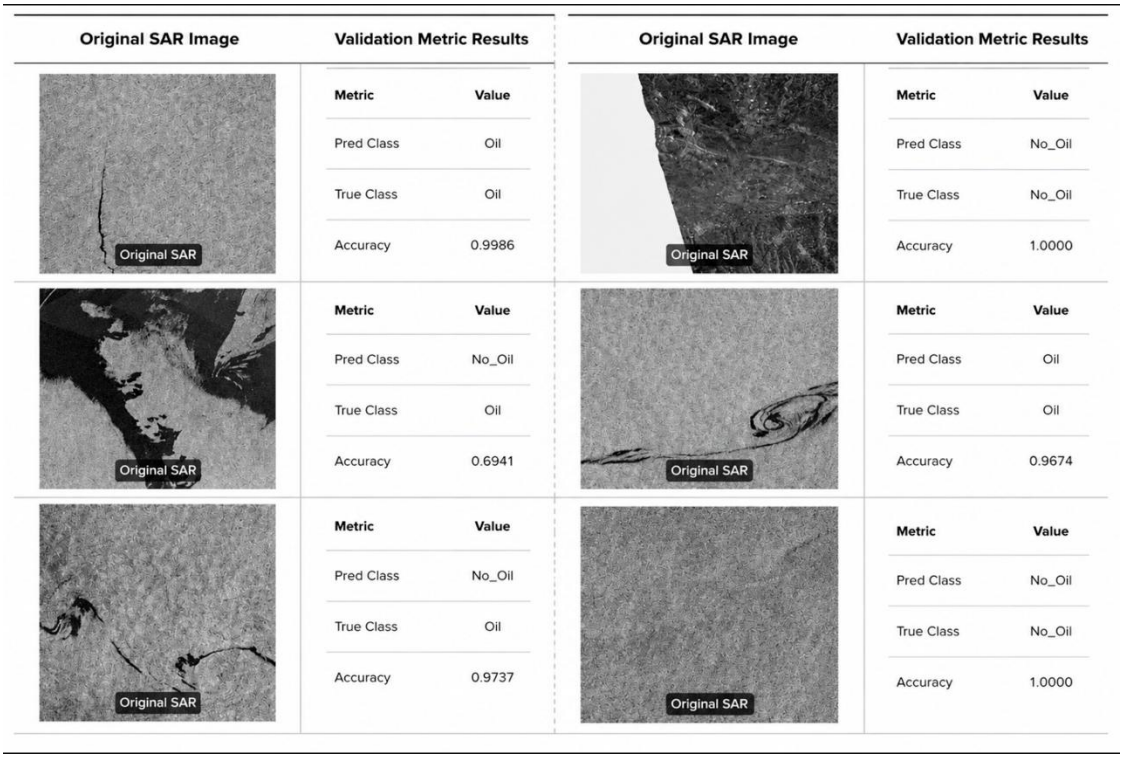


Figure 11.2: Validation results showing original SAR images and corresponding classification metrics

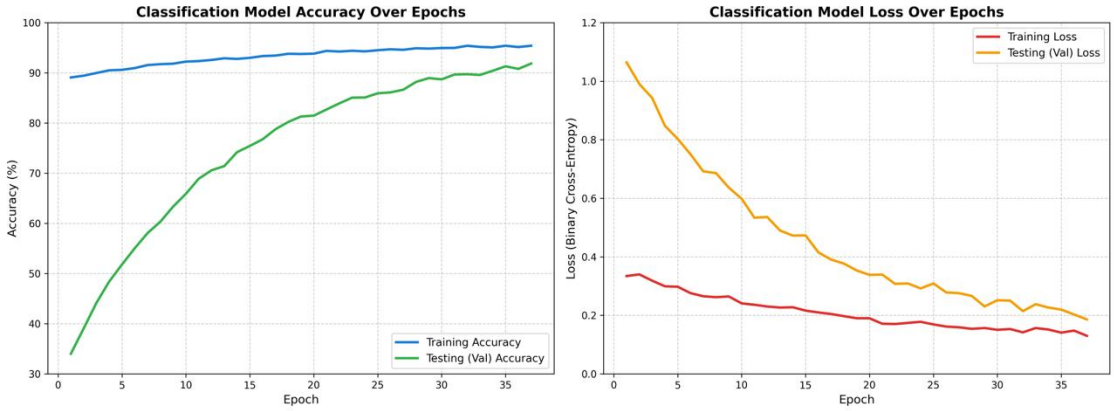


Figure 11.3: Classification Model Accuracy and Loss Curves

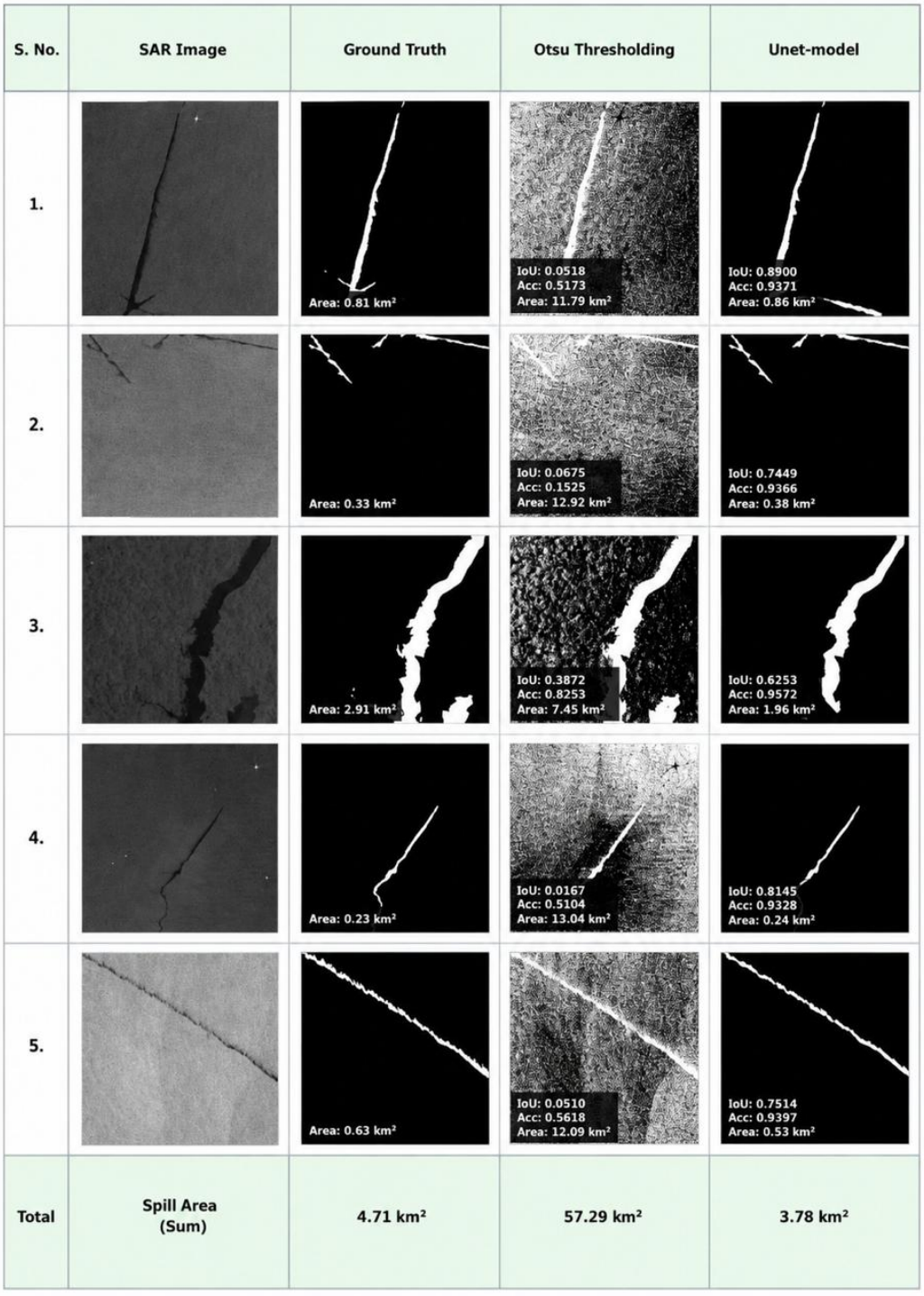


Figure 11.4: Performance Comparison of SAR Oil Spill Segmentation Methods

Comparison Between U-Net and Otsu Thresholding

Figure 11.4 shows the comparison between the traditional Otsu thresholding method [21] and the proposed U-Net segmentation model for oil spill detection using SAR images. It can be observed that Otsu thresholding produces several false detections due to noise and

variations in pixel intensity present in SAR images. As a result, the segmented regions are often fragmented and include unwanted background areas.

In contrast, the U-Net model generates segmentation masks that are much closer to the ground truth masks. The predicted boundaries are smoother and more accurate, resulting in higher IoU and accuracy values. Furthermore, the spill area estimated by the U-Net model closely matches the actual spill area, whereas Otsu thresholding tends to overestimate the contaminated region. As shown in Figure 11.3, the cumulative spill area obtained using Otsu thresholding is 57.29 km^2, which is significantly higher than the ground truth value of 4.71 km^2. The proposed U-Net model estimates a total spill area of 3.78 km^2, demonstrating much better agreement with the actual oil spill regions.

Overall, the experimental results indicate that the U-Net model provides superior segmentation performance compared to conventional Otsu thresholding and is more suitable for accurate oil spill detection and area estimation in SAR imagery.

CHAPTER 12

PROJECT PLAN

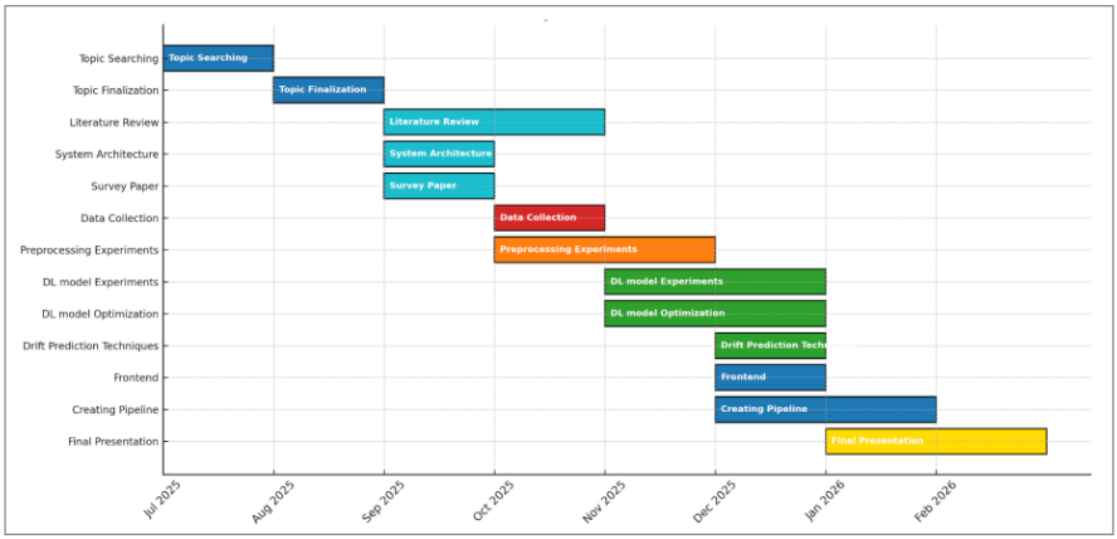


Figure 12.1: Project Plan

CHAPTER 13

CONCLUSIONS AND FUTURE WORK

13.1 CONCLUSION

This project presented an intelligent framework for Oil Slick Detection and Drift Prediction using Sentinel-1 SAR satellite imagery. A deep learning-based classification model was developed to automatically identify oil slicks and distinguish them from look-alike phenomena commonly observed in marine environments. A U-Net segmentation model was further employed to accurately delineate the spatial extent of detected oil spills.

Following detection and segmentation, the geographical coordinates of the oil slick were extracted from the SAR GeoTIFF image and used as input to the OpenDrift framework. A Lagrangian particle-based simulation was performed to predict the future trajectory and spread of the oil spill under the influence of environmental factors such as ocean currents and wind conditions.

The developed system provides an end-to-end workflow for oil spill monitoring, combining satellite image analysis, deep learning, geospatial processing, and drift simulation. The generated outputs, including oil slick masks, drift trajectories, and animations, can assist environmental agencies and maritime authorities in planning effective response and containment operations.

13.2 FUTURE WORK

Although the proposed system demonstrates promising results, several improvements can be incorporated in future work:

- Integration of real-time environmental datasets from sources such as Copernicus Marine Services and HYCOM for more accurate drift prediction.
  ‘‘‘
- Development of advanced deep learning architectures, including Transformer-based and attention-based models, to improve oil slick detection performance.
- Incorporation of weathering processes such as evaporation, dispersion, and emulsification to enhance the realism of oil spill simulations.

- Extension of the system to support multi-temporal satellite imagery for continuous monitoring of oil spill evolution.
- Deployment of the complete framework as a cloud-based decision support system for real-time marine pollution monitoring and response. “‘

13.3 ANALYSIS

Over 50 models were developed and evaluated using different patch sizes (16×16, 32×32), dropout rates, learning rates, and loss weights. For oil spill classification, a CNN was trained using binary cross-entropy loss and Adam optimization to distinguish real oil spills from lookalikes such as low-wind areas and biogenic films.

A major finding was that combining dual-polarization SAR data (VV and VH) significantly improved performance. While VV captures changes in sea surface roughness caused by oil, it is susceptible to false positives from low-wind regions. The addition of VH provided complementary texture information, reducing misclassifications and improving robustness.

To address SAR speckle noise, intensity clipping (VV: [-35, 5] dB, VH: [-40, 0] dB) and spatial dropout were applied. These preprocessing and regularization techniques stabilized training and achieved approximately 96

For segmentation, a U-Net architecture was employed. Its encoder-decoder design with skip connections enabled accurate recovery of fine oil slick boundaries and thin filament structures that would otherwise be lost during downsampling. Compared to traditional Otsu thresholding, U-Net provided better contextual understanding and generated more cohesive masks, achieving a 91.5

Finally, morphological post-processing using 5×5 elliptical opening and closing operations removed isolated noise and filled small gaps within the segmented regions, producing cleaner masks suitable for reliable oil spill area estimation.

REFERENCES

[1] C. Popa, D. Atodiresei, A. Toma, V. Dobref, and J. Vatamanu, "Solutions for Modelling the Marine Oil Spill Drift," *Environments*, vol. 12, no. 4, p. 132, Apr. 2025, doi: 10.3390/environments12040132.

[2] C. Dearden, T. Culmer, and R. Brooke, "Performance Measures for Validation of Oil Spill Dispersion Models Based on Satellite and Coastal Data," *IEEE Journal of Oceanic Engineering*, vol. 47, no. 1, pp. 126–140, Sep. 2021, doi: 10.1109/joe.2021.3099562.

[3] P. Berens, "Introduction to Synthetic Aperture Radar (SAR)," 2006. Available: apps.dtic.mil/sti/tr/pdf/ADA470686.pdf.

[4] F. Mahdikhani and M. Hassannejad Bibalan, "Detection of Oil Slicks in SAR Satellite Images Using Otsu-Bradley's Thresholding Method," *Majlesi Journal of Electrical Engineering*, vol. 19, no. 2, Jun. 2025, doi: 10.57647/j.mjee.2025.1902.24.

[5] J. Xu *et al.*, "Oil Slick Identification in Marine Radar Image Using HOG, Random Forest and PSO," *IEEE Geoscience and Remote Sensing Letters*, vol. 21, pp. 1–5, Jan. 2024, doi: 10.1109/lgrs.2024.3431043.

[6] E. Kalogirou *et al.*, "Oil Spill Detection Using Convolutional Neural Networks and Sentinel-1 SAR Imagery," *The International Archives of the Photogrammetry, Remote Sensing and Spatial Information Sciences*, vol. XLVIII-G-2025, pp. 757–764, Jul. 2025, doi: 10.5194/isprs-archives-xlviii-g-2025-757-2025.

[7] H. Guo, G. Wei, and J. An, "Dark Spot Detection in SAR Images of Oil Spill Using SegNet," *Applied Sciences*, vol. 8, no. 12, p. 2670, Dec. 2018, doi: 10.3390/app8122670.

[8] O. Ronneberger, P. Fischer, and T. Brox, "U-Net: Convolutional Networks for Biomedical Image Segmentation," 2015. Available: arxiv.org/pdf/1505.04597.pdf.

[9] D. Xiang, Y. Lu, D. Guan, G. Li, J. Cheng, and B. Li, "Oil Spill Detection in PolSAR Imagery Using Composite Scattering Power Entropy and Multi-Scale Hybrid Feature Fusion Network," *IEEE Journal of Selected Topics in Applied Earth Observations and Remote Sensing*, pp. 1–20, Jan. 2025, doi: 10.1109/jstars.2025.3570697.

[10] R. M. Haralick, K. Shanmugam, and I. Dinstein, "Textural Features for Image Classification," *IEEE Transactions on Systems, Man, and Cybernetics*, vol. SMC-3, no. 6, pp. 610–621, Nov. 1973, doi: 10.1109/tsmc.1973.4309314.

[11] A. S. Mahmoud, S. A. Mohamed, R. A. El-Khoriby, H. M. AbdelSalam, and I. A. El-Khodary, "Oil Spill Identification Based on Dual Attention U-Net Model Using Synthetic Aperture Radar Images," *Journal of the Indian Society of Remote Sensing*, vol. 51, no. 1, pp. 121–133, Nov. 2022, doi: 10.1007/s12524-022-01624-6.

[12] K.-F. Dagestad, J. Röhrs, Ø. Breivik, and B. Ådlandsvik, "OpenDrift v1.0: A Generic Framework for Trajectory Modelling," *Geoscientific Model Development*, vol. 11, no. 4, pp. 1405–1420, Apr. 2018, doi: 10.5194/gmd-11-1405-2018.

[13] M. De Dominicis, N. Pinardi, G. Zodiatis, and R. Lardner, "MEDSLIK-II: A Lagrangian Marine Surface Oil Spill Model for Short-Term Forecasting – Part 1: Theory," *Geoscientific Model Development*, vol. 6, no. 6, pp. 1851–1869, Nov. 2013, doi: 10.5194/gmd-6-1851-2013.

[14] D. Liu, Y. Li, and L. Mu, "Parameterization Modeling for Wind Drift Factor in Oil Spill Drift Trajectory Simulation Based on Machine Learning," *Frontiers in Marine Science*, vol. 10, Jul. 2023, doi: 10.3389/fmars.2023.1222347.

[15] B. G. Gautama, N. Longepe, R. Fablet, and G. Mercier, "Assimilative 2-D Lagrangian Transport Model for the Estimation of Oil Leakage Parameters from SAR Images: Application to the Montara Oil Spill," *IEEE Journal of Selected Topics in Applied Earth Observations and Remote Sensing*, vol. 9, no. 11, pp. 4962–4969, Nov. 2016, doi: 10.1109/jstars.2016.2606110.

[16] W. J. Guo and Y. X. Wang, "A Numerical Oil Spill Model Based on a Hybrid Method," *Marine Pollution Bulletin*, vol. 58, no. 5, pp. 726–734, May 2009, doi: 10.1016/j.marpolbul.2008.12.015.

[17] W. Shao *et al.*, "Influence of Sea Surface Waves on Numerical Modeling of an Oil Spill: Revisit of Symphony Wheel Accident," *Journal of Sea Research*, vol. 201, pp. 102529–102529, Aug. 2024, doi: 10.1016/j.seares.2024.102529.

[18] K. Kampouris, V. Vervatis, J. Karagiorgos, and S. Sofianos, "Oil Spill Model Uncertainty Quantification Using an Atmospheric Ensemble," *Ocean Science*, vol. 17, no. 4, pp. 919–934, Jul. 2021, doi: 10.5194/os-17-919-2021.

[19] P. Keramea *et al.*, "Satellite Imagery in Evaluating Oil Spill Modelling Scenarios for the Syrian Oil Spill Crisis, Summer 2021," *Frontiers in Marine Science*, vol. 10, Oct. 2023, doi: 10.3389/fmars.2023.1264261.

[20] S. Fan, L.-Y. Oey, and P. Hamilton, "Assimilation of Drifter and Satellite Data in a Model of the Northeastern Gulf of Mexico," *Continental Shelf Research*, vol. 24, no. 9, pp. 1001–1013, May 2004, doi: 10.1016/j.csr.2004.02.013.

[21] F. Yu, W. Sun, J. Li, Y. Zhao, Y. Zhang, and G. Chen, "An Improved Otsu Method for Oil Spill Detection from SAR Images," *Oceanologia et Hydrobiologia*, vol. 59, no. 3, pp. 311–317, Jul. 2017, doi: 10.1016/j.oceano.2017.03.005.

[22] K. Li, H. Yu, Y. Xu, and X. Luo, "Detection of Oil Spills Based on Gray Level Co-occurrence Matrix and Support Vector Machine," *Frontiers in Environmental Science*, vol. 10, Dec. 2022, doi: 10.3389/fenvs.2022.1049880.

[23] M. Reinan *et al.*, "SAR Oil Spill Detection System Through Random Forest Classifiers," *Remote Sensing*, vol. 13, no. 11, p. 2044, May 2021, doi: 10.3390/rs13112044.

[24] L. Lopez, M. Moctezuma, and F. Parmiggiani, "Oil Spill Detection Using GLCM and MRF" Nov. 2005, doi: 10.1109/igarss.2005.1526349.

[25] R. Trujillo-Acatitla, J. Tuxpan-Vargas, C. Ovando-Vázquez, and E. Monterrubio-Martínez, "Sentinel-1 SAR Oil spill image dataset for train, validate, and test deep learning models. Part I", Zenodo, 2024, doi: 10.5281/zenodo.8346860.

[26] J. M. Macharia, F. K. Ngetich, and C. A. Shisanya, "Comparison of Satellite Remote Sensing Derived Precipitation Estimates and Observed Data in Kenya," *Agricultural and Forest Meteorology*, vol. 284, Art. no. 107875, Apr. 2020, doi: 10.1016/j.agrformet.2019.107875

APPENDICES

TOOLS USED

Different techniques have been applied during the analysis of satellite based SAR imagery.

Firstly, the dataset used for both training and testing purposes was sourced from the Zenodo open access repository. This data contains Sentinel-1 Synthetic Aperture Radar (SAR) images in GeoTIFF format with dual-polarization bands VV and VH, which are required to detect the oil spills and differentiate them from similar-looking objects. Other SAR images were available in platforms like the Copernicus Open Access Hub and other sources of satellite imagery.

Python was the main programming language used to implement the system. The deep learning models that can be used to detect and segment oil spills were implemented in TensorFlow and Keras.

Geospatial data processing and SAR images processing were done using Rasterio and SNAP which helped in reading and processing GeoTIFF images.Image processing operations were handled using tools such as NumPy and OpenCV. Visualization of SAR images, segmentation masks, and drift prediction was done using Matplotlib.

For drift prediction, data such as wind speed, ocean current, and temperature were collected through Copernicus weather APIs. These data were then added to OpenDrift simulation tool to simulate the oil spill trajectory.

Analysis and experiments were done using Jupyter notebook while code development and debugging were done using Visual Studio Code. GitHub was used for project management.

The use of AI tools such as ChatGPT and GitHub Copilot was made during the development process. ChatGPT was employed in helping to comprehend the concepts related to SAR image processing, bug tracking in the code, documentation creation, and technical assistance. GitHub Copilot was used in the capacity of a coding assistant for the purpose of speeding up development.

DrillBit

AI Content Detection Report
Submission ID: 175862

AI TEXT DETECTED (%)
Percentage of content showing AI-generated patterns

22.49

Matched 3141 words showing AI-pattern signal

POSITION ON SCALE
Where this score falls on a 0 - 100% range

0% 100%

22.49 %

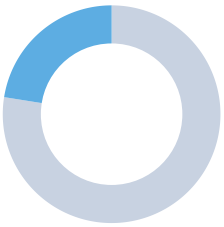
01 Submission Information

Document metadata

Author Name	Paras Ningune
Title	OIL SLICK DETECTION AND DRIFT PREDICTION USING SATELLITE IMAGERY
Paper/Submission ID	175862
Submitted By	paras.ningune01@gmail.com
Submission Date	2026-06-26
Document Type	Project Work
Language	English
Text [Pages, Sentences, Words]	[69, 650, 13661]

02 Content Composition

Colors match in-document highlighting



Human Written — 77.51 %
Original or non-flagged content

AI Text — 22.49 %
Highlighted in blue in the document

Excluded — 0.00 %
Manually excluded content

SCAN

AI INTEGRITY HELP
Guides, policy, and best practices.

► **AI Help Centre**
drillbitglobal.com/ai-content-detection

► **Detection Methodology**
drillbitglobal.com/ai-method

DrillBit

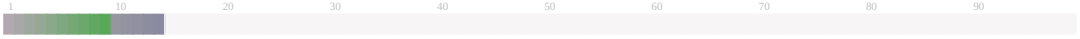
The Report is Generated by DrillBit Plagiarism Detection Software

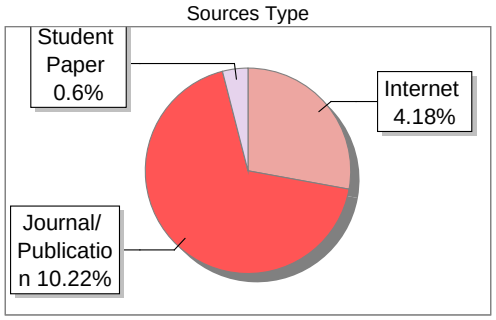
Submission Information

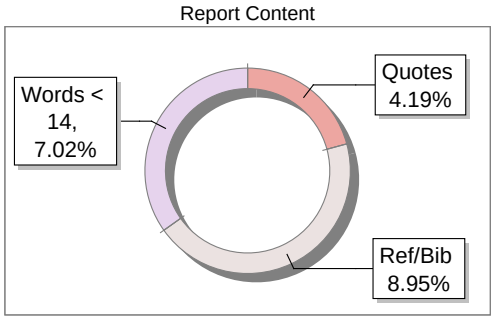
Author Name	Paras Ningune
Title	OIL SLICK DETECTION AND DRIFT PREDICTION USING SATELLITE IMAGERY
Paper/Submission ID	175862
Submitted by	paras.ningune01@gmail.com
Submission Date	2026-06-26
Total Pages, Total Words	69, 13661
Document type	Project Work

Result Information

Similarity **15 %**







Exclude Information

Quotes	Not Excluded
References/Bibliography	Not Excluded
Source: Excluded < 14 Words	Not Excluded
Excluded Source	**0 %**
Excluded Phrases	Not Excluded

Database Selection

Language	English
Student Papers	Yes
Journals & publishers	Yes
Internet or Web	Yes
Institution Repository	Yes

A Unique QR Code use to View/Download/Share Pdf File

Grenze International Journal of Engineering and Technology, January Issue

Oil Slick Detection and Drift Prediction using Satellite Imagery

Paras Ningune[1], Sakshi Patil[2], Soham Mane[3] and Siddhant Pote[4]
[1-4]Department of Information Technology, Savitribai Phule Pune University, Pune, India
Email: paras.ningune01@gmail.com, psakshid24@gmail.com, manesoham219@gmail.com, siddhantpote20@gmail.com

***Abstract*— This study introduces an intelligent, automated system that uses high-resolution SAR imagery together with environmental data for detecting and tracking oil slicks. By combining dark spot detection, texture and semantic analysis and deep learning, it minimizes false alarms and accurately predicts slick movement based on wind, currents and tides for real-time marine monitoring.**

***Index Terms*— Oil Slick Detection, Satellite Imagery, SAR, Drift Prediction, Remote Sensing.**

I. INTRODUCTION

The thin layers of oil floating on the ocean's surface which are petroleum-based are known as Oil Slicks. They have grown as a serious environmental issue as even a tiny amount of oil can spread quickly over a large ocean area. While some occur naturally as oil seepage through the seafloor, most oil slicks are a result of human attempts such as tanker collisions, pipeline ruptures, offshore drilling operations, and discharge of ship wastes.
These slicks are one of the major causes of damage to the ocean environment. One layer that seems to be more affected is the one where microscopic phytoplankton, the basis of the ocean food web, live. The phytoplankton are being hindered from getting oxygen and sunlight because of the oily coating on the water. To make matters worse, marine life takes in the oil either through their food or when they breathe while seabirds, on the other hand, lose their waterproof feature and suffer a kind of damage that is not only severe but also irreversible.
Since Synthetic Aperture Radar (SAR) satellites such as Sentinel-1 and Landsat can keep track of the oceans regardless of the weather, time of the day, etc., they are being utilized to bring fast help in detecting spills which are eventually released into the sea. Oil slicks on the ocean surface look quite bright in SAR images, but one needs to be cautious about natural films and very calm sea surface, because both of them can show up as if it was an oil spill, and hence making detection hard. Earlier methods were based on very simple threshold-based functions and also people visually inspecting images for locations of oil spills. Later on, machine learning techniques like SVMs and Random Forests were used, thereby raising detection accuracy. The present state of art in this field is represented by deep learning architectures like CNNs, U-Net, and SegNet, which in theory should give even better results, however, issues such as false positives and shortage of labeled data still remains.
Finding a slick is only half the challenge. The next question is: where will it go? Drift prediction models try to answer this. Lagrangian models work on individual oil particles, while Eulerian models calculate oil concentration across grids [1]. Factors such as wind, currents, spreading, evaporation, and even how oil breaks down over time are required by these models. SAR imagery, hydrodynamic models and sensor data from buoys or coasts combined can produce predictions far more reliable.

Grenze ID: 01.GIJET.12.1.163_2

Essentially, a whole pipeline that connects detection to prediction i.e. satellites gather the images, deep learning models detect whether it is a spill or not, drift models predict the movement, thus the cleanup crews can deploy barriers or recovery vessels in the right places.
There are still quite a few obstacles on the way. Look-alikes can raise a major problem in detecting, while drift models suffer from constantly changing environmental conditions [1, 3]. Sensor fusion (drones plus satellites), bigger labeled datasets for detection model training, and hybrid systems that can utilize human knowledge and machine efficiency might be the next big breakthroughs [1, 7].

II. Role of Satellites

Ship patrols and flying surveys were the primary sources of data on oil spills, where and how extensive they were. The way these are done is highly costly and inefficient, though. Ship patrols were only able to scan a limited area of the ocean at any given time, and it took many people, fuel, and planning time. While it can be done by flying surveys, it takes skilled manpower, specialist aircraft, and fine weather conditions.
One of the major issues with these techniques was that they were unable to scan a large area over a long period. It is not feasible to keep watching areas with ships and aircraft which can inspect a particular area at particular moments [1]. Oil slicks disperse rapidly on account of wind and ocean currents, and hence their early identification is extremely critical. If we procrastinate, then they will spread everywhere which will make it harder for us to clean which can lead to even worse financial and environmental damage [5].
The development of satellite technologies has provided us with much better solutions to these issues. Satellites can accomplish the task which is performed by conventional methods again and again in which oceans can be surveyed. They are able to identify spills in the middle of nowhere which is difficult to access. They are able to do so because satellites capture high-resolution images regardless of weather [3]. Because they can be tracked in real-time, the public is made aware of these occurrences and thus they can react to oil spills in a more efficient and quicker manner.
With the assistance of satellite surveillance that is constantly on and scans very large geographic areas, oil spill detection and response became feasible. Due to continuous monitoring it has made experts monitor the spread movements of oil with time because satellites can capture thousands of kilometers of area in a single pass [1,10]. This innovation has significantly helped us in preservation of marine ecosystems by coordination responses and improving early warning systems.

III. Sentinel-1 Polarization Modes

In SAR imaging, the term polarization refers to the orientation of the transmitted and received radar with either a horizontal
(H) or vertical (V) polarization. Polarizations can be arranged, depending on how the radar transmits and receives, in order to separate information about the surface, and the scattering characteristics of the surface.
In single polarization modes (VV or HH), the radar transmits and receives in the same direction. For example, VV means the radar transmits a vertically polarized wave and receives the vertically polarized echo as well. An HH signal, both the transmission and reception occur in the horizontal direction. Accordingly, these modes are simpler and do not require storing a significant amount of data, but they do not also provide much information about the surface.

TABLE I: COMPARISON OF SATELLITES

Satellite	Sensor Type	Spatial Resolution	Revisit Time
Sentinel-1	C-Band SAR	~10 m	6–12 days
Sentinel-2	Multispectral Optical	10–60 m	5 days
RADARSAT	C-Band SAR	3–100 m	24 hrs
TerraSAR-X	X-Band SAR	~1–3 m	Depends on orbit and tasking
RISAT-2	X-Band SAR	~1 m	Depends on orbit and tasking
Cartosat-2	Panchromatic Optical	~1–3 m	5 days

Dual polarization modes (like HV or VH) offer not only mode information; they also have richer data. That means the radar transmits in one polarization (either horizontal or vertical) and receives in both co-polarized and cross-polarized directions. The advantage of dual polarization is that it provides more information about how the radar wave interacts with the surface materials. Therefore, dual polarization SAR provides better classification results, and more reliability, making it more effective than single polarization for tasks like environmental monitoring and oil spill detection.

TABLE II: SENTINEL-1 POLARIZATION MODES

Polarization	Meaning	Modes
VV	Transmit Vertical, Receive Vertical	Single Polarization
HH	Transmit Horizontal, Receive Horizontal	Single Polarization
HV	Transmit Horizontal, Receive Vertical	Dual Polarization (HH + HV)
VH	Transmit Vertical, Receive Horizontal	Dual Polarization (VV + VH)

IV. CLASSICAL IMAGE PROCESSING AND THRESHOLDING

To reduce false positives early, oil slick detection techniques like SAR image processing and thresholding were used to extract dark spots, followed by geometric and textural analysis. Because of their explainability and low computational cost - especially when preprocessing is used - these methods, despite their simplicity, continue to be popular and effective.

Otsu's method [4] is a well-known global thresholding technique that uses image histogram analysis to automatically divide an image into foreground and background. It chooses a threshold that optimizes the difference between the object and the background, or inter-class variance. Because it doesn't require any user parameters, its primary advantages are automation and simplicity. However, because it applies a single global threshold, it performs best when the image has a bimodal intensity distribution and tends to fail in the presence of noise, uneven illumination, or non-bimodal histograms.

Bradley's approach [4], however, uses a local adaptive thresholding technique. It determines the local mean intensity in the neighborhood surrounding each pixel rather than employing a single global threshold, and then sets the threshold marginally below this mean. This enables it to efficiently manage changes in lighting and shadows. However, because thresholds need to be calculated for several local regions, it is computationally more costly. The choice of window size is also crucial; a window that is too big decreases adaptability, while one that is too small increases sensitivity to noise.

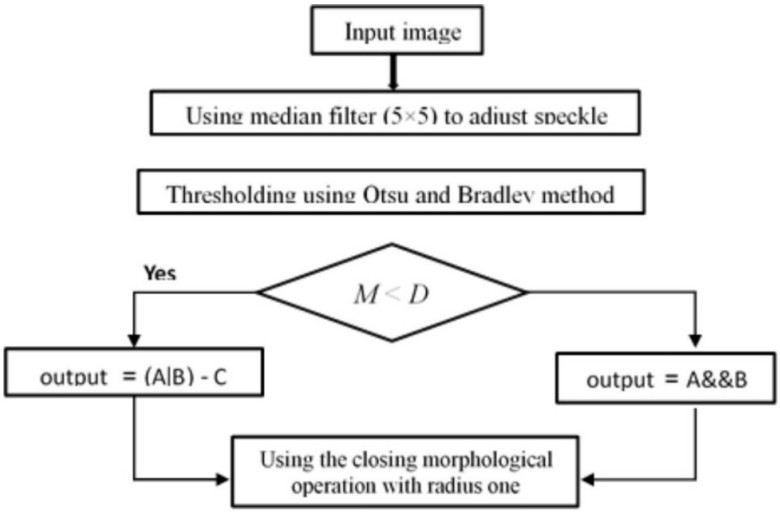


Figure 1: Flowchart for Otsu-Bradley's thresholding [4].

V. MACHINE LEARNING APPROACHES

Machine learning (ML) approaches [5] have become more and more popular for dark spot detection in SAR images because of their capability to handle high dimensional feature spaces. Traditional machine learning classifiers such as K-Nearset Neighbour, Support Vector Machines (SVM), Random Forest, and Decision Trees are most widely used to distinguish oil slicks from their look-alikes. Features such as shape descriptors, texture features and polarimetric parameters are used by these models [5].

Pre-processing methods such as thresholding or textural analysis are usually applied for identifying possible dark spots. The extracted features are then inputted to various ML classifiers to classify which has been trained using labeled datasets of oil and non-oil regions. Sometimes there is an ambiguity where oil slicks and look-alike overlap but are handled by using fuzzy C-means.

Detection accuracy can be improved by using machine learning (ML) methods. It also reduces false positives caused by ancillary information. As a result ML models are becoming an essential part of modern oil spill monitoring systems, offering a more reliable and scalable alternative to traditional threshold-based techniques.

VI. Deep Learning Approaches

A. Convolution Neural Network

Convolution Neural Networks (CNNs) have become very successful methods due to their ability to detect oil slips because of their capability to extract. The research indicates that because of their high accuracy in feature extraction, high precision and recall and minimizing the false positives have led to their increased reliability [7, 8, 11]. CNNs are better than traditional approaches such as dark spot detection, textural analysis and machine-learning approaches. The performance of detection has been enhanced further because of hybrid architecture that combines boosting SVMs and encoder-decoder networks [5].

The detection frameworks based on CNN are primarily trained and tested using standardized techniques like 70-20-10 data splitting and optimization techniques such as Adam with binary cross-entropy loss. Priority performance metrics such as accuracy, precision, recall, and F1-score are considered to measure the robustness of the model. The confusion metrics are commonly applied for measuring false positives and false negatives to ensure the system is reliable for prolonged environmental usage.

Model performance is mainly measured using indicators such as accuracy, precision, recall, and F1-score, whose validation is done through confusion matrices. Model robustness and generalization across the world marine environment can be boosted by expanding the training set size to cover diverse spills scenarios, sea conditions and combining different satellite sensors [1, 3].

B. U-Nets

Convolutional Neural Networks (CNN's) and its types have become particularly famous and important due to its ability to capture oil slick images from satellites. Particularly U-Net due to its encoder-decoder architecture with skip connections, stands out as its architecture helps preserve spatial details during image reconstruction. Mainly designed for biomedical image segmentation, U-Net has proved to be highly effective in portraying the fine oil spill boundaries in SAR imagery. However, U-Net models sometimes face challenges in handling simple spill shapes or hard to interpret regions like low- wind zones and biogenic films as these regions fail to provide broader contextual information which is required for precise segmentation [8].

Attention mechanisms were incorporated into U-Net models in order to address the issues and lower the detection of false positives. Attention-based U-Net models improve the detection of intricate spill shapes by reducing background noise from winds or waves in addition to identifying significant regions. There are various versions of U-Net which are available which used both i.e., space and channel which is used to capture attention of seas and features which helps us to increase the performance as compared to using traditional models which are not very useful and inefficient. The metrics used by U-Net models are f1-score, accuracy, recall and IOU.

In previous forms, U-Nets were primarily concerned with tiny local image features. For this reason, they may get confused between actual oil spills and other darker regions in SAR images such as calm water or natural seepage. Dual Attention U-Nets helps us to capture both global and local fine detail images which helps us to improve their reliability, flexibility and effectiveness under various different ocean conditions in consideration to ancillary data. Due to its development it has helped us to significantly advance in the field of Al, which resulted in giving us more practical and accurate real-world use.

C. SegNets

SegNet was first designed for autonomous driving but is now also used to detect dark spots in SAR images of oil spills. It uses an encoder-decoder structure based on a the VGG-16 model. In this architecture the encoder heps extract crucial features from the image using convolution and pooling layers, while the decoder builds the image using up-sampling layers to make pixel-wise predictions.

The defining attribute of SegNet is that it remembers the positions (pooling indices) from the encoder, which helps the decoder accurately restore the object boundaries. This allows SegNet to capture fine and irregular shapes such as oil spill regions more precisely.

Compared to older methods like thresholding or region-based approaches, SegNet performs better in noisy SAR images with uneven brightness or blurry edges. It maintains spatial details and understands the overall context of the image, making it a strong and reliable model for detecting oil spills in complex marine environments [7].

VII. Drift Prediction Models

Spill drift prediction is essential for forecasting the future path of an oil slick, its spread extent, and potential impact areas. The drift behavior is primarily influenced by surface currents, wind forcing, Stokes drift, and oil weathering. To reproduce these processes, several modeling frameworks Lagrangian, Eulerian, Hybrid/Ensemble, and Data Assimilation are widely used.

Lagrangian models simulate an oil slick as a collection of small virtual particles that drift with ocean currents, wind, and waves. Random-walk terms are included to represent turbulence. Tools such as NOAA's GNOME and MEDSLIK-II are widely used for short-term oil spill forecasting [13]. The accuracy of these models depends on environmental factors such as the Wind Drift Factor (WDF). Recent studies apply machine learning methods, including Support Vector Regression and Deep Learning, to dynamically estimate WDF and improve simulation accuracy [14]. The initial distribution of these virtual particles is often based on Sentinel-1 SAR data, which also helps verify the predicted drift paths [16].

Eulerian models, in contrast, treat oil as a continuous field influenced by advection–diffusion equations. They incorporate physical and chemical processes such as spreading, evaporation, emulsification, dissolution, and sedimentation [17]. These models are particularly suited for large-scale and long-term simulations.

Hybrid systems combine both Eulerian and Lagrangian methods using particles to represent scattered oil slicks and grids for larger, continuous slicks. Hybrid models can accurately predict both trajectories and concentration fields. Ensemble techniques operate by running multiple simulations with varying inputs of winds, currents, and other conditions. Studies using MEDSLIK-II and ECMWF ensembles have generated probabilistic risk maps, improving uncertainty quantification in coastal regions [13, 11].

Data assimilation techniques integrate real-time observations (such as SAR, optical, and drifter data) with drift models to enhance prediction accuracy [16]. Methods such as the Ensemble Kalman Filter improve short-term forecasting and minimize errors beyond 24-48 hours. Recent studies have also employed adaptive machine learning models that correct their own errors in real-time leveraging SAR updates to keep model parameters tuned [14, 16].

Lagrangian models are typically preferred for fast trajectory forecasting, while Eulerian models are used for large-scale and long-term oil spill simulations, including detailed fate analysis. Hybrid and ensemble frameworks improve reliability through uncertainty quantification, and data assimilation ensures adaptive, real-time correction. Together, these models supported by Sentinel-1 SAR data form the backbone of modern oil slick drift prediction and response systems [16].

VIII. Datasets

Researchers have utilized various satellite and radar datasets to analyze the movement and development of oil slicks across marine regions. TerraSAR and TanDEM satellite data were used to study oil slick evolution in the North Sea [1]. With a short 13-hour revisit time, these satellites enabled tracking of slicks at different stages of formation. The observations closely matched NOAA's GNOME model predictions, enhancing the understanding of short-term oil slick behavior. The high-resolution imagery further improved the accuracy of oil spill monitoring and ocean drift forecasting.

Xu [5] adopted a different approach using an X-band marine radar installed on the Yukun training vessel from Dalian Maritime University. This ship-based system provided continuous, real-time tracking without the time gaps associated with satellite observations. The radar images were divided into smaller sections and analyzed using machine learning methods such as Histogram of Oriented Gradients (HOG) and Random Forest (RF), optimized with Particle Swarm Optimization (PSO). The results demonstrated that low-cost, localized monitoring is effective for nearby areas and that ship-based radar can serve as a reliable backup to satellites for detecting regional oil spills.

The newly developed ISPRS Sentinel-1 SAR dataset [6], covering the Mediterranean Sea, includes 695 images from 50 confirmed oil spill events recorded between 2014 and 2019. Each image was calibrated, filtered to reduce speckle noise, and converted from decibel to power values. The processed images were then divided into 256 × 256 patches with matching ground-truth masks. Experts split the data into training, validation, and test sets for deep learning experiments using Convolutional Neural Networks (CNNs). With a balanced mix of spill and clean samples, this open dataset provides
a strong, standardized benchmark for developing models and addressing data imbalance issues in SAR-based oil spill detection.

Another important resource is the Radarsat-2 full PolSAR dataset [9], which includes both intentional spills in the North Sea and natural spills in the Gulf of Mexico. Owing to its high-resolution polarimetric data, this dataset helps distinguish real oil slicks from look-alike features such as biological films or calm water areas. It also supports the development of deep learning models such as MSHFFNet, enabling more accurate and robust oil spill detection.

The Corsican Spill dataset [13], derived from Sentinel-1 SAR imagery following a tanker spill, contains two weeks of time-series images used to model oil drift and coastal impacts through simulations such as GNOME and MEDSLIK-II. Additionally, the Deep-SAR Oil Spill (SOS) dataset consists of more than 8,000 labeled patches from large-scale events, including the Deepwater Horizon and Al-Khafji spills. One of the most valuable open-access datasets, it has been widely used to train deep learning models such as CBD-Net, significantly improving automated oil spill detection in terms of accuracy, precision, reliability, and resilience.

IX. CONCLUSIONS

Oil slick detection and drift prediction are vital for safeguarding marine ecosystems, guiding emergency response, and enforcing maritime regulations. Over the years, methods have evolved significantly from conventional interpretation and manual thresholding to advanced machine learning and deep learning techniques and frameworks. Otsu and Bradley are computationally efficient, but they often fail in complex sea states due to look-alike features. Machine learning and deep learning methods techniques have improved automation but they still heavily depend on handcrafted features or labeled datasets. Deep Learning has revolutionized the field, with CNNs, U-Net, and SegNet models achieving higher accuracy and by learning directly from large-scale SAR datasets.

Similarly, drift prediction has progressed from conventional methods like Lagrangian and Eulerian models to hybrid, ensemble, and data assimilation in order to capture better nonlinear ocean dynamics. The use of SAR images, optical, and environment data has increased the reliability, enhancing real-time monitoring and forecasting of oil slick.

Despite such advances in the technology some challenges arise and remain such as limited availability of datasets, high computational power and lack of standardised metrics. Future studies and research focus on hybrid detection, physics-informed AI, and global benchmark datasets. Addressing these gaps, next-generation systems can deliver scalable, real-time, and globally deployable solutions for marine oil spill monitoring and management.

REFERENCES

[1] C. Popa, D. Atodiresei, A. Toma, V. Dobref, and J. Vatamanu, “Solutions for Modelling the Marine Oil Spill Drift,” Environments, vol. 12, no. 4, p. 132, Apr. 2025, doi: 10.3390/environments12040132.

[2] C. Dearden, T. Culmer, and R. Brooke, “Performance Measures for Validation of Oil Spill Dispersion Models Based on Satellite and Coastal Data,” IEEE Journal of Oceanic Engineering, vol. 47, no. 1, pp. 126–140, Sep. 2021, doi: 10.1109/joe.2021.3099562.

[3] P. Berens, “Introduction to Synthetic Aperture Radar (SAR),” 2006. Available: apps.dtic.mil/sti/tr/pdf/ADA470686.pdf.

[4] F. Mahdikhani and M. Hassannejad Bibalan, “Detection of Oil Slicks in SAR Satellite Images Using Otsu- Bradley’s Thresholding Method,” Majlesi Journal of Electrical Engineering, vol. 19, no. 2, Jun. 2025, doi: 10.57647/j.mjee.2025.1902.24.

[5] J. Xu et al., “Oil Slick Identification in Marine Radar Image Using HOG, Random Forest and PSO,” IEEE Geoscience and Remote Sensing Letters, vol. 21, pp. 1–5, Jan. 2024, doi: 10.1109/lgrs.2024.3431043.

[6] E. Kalogirou et al., “Oil Spill Detection Using Convolutional Neural Networks and Sentinel-1 SAR Imagery,” The International Archives of the Photogrammetry, Remote Sensing and Spatial Information Sciences, vol. XLVIII-G- 2025, pp. 757–764, Jul. 2025, doi: 10.5194/isprs-archives-xlviii-g-2025-757-2025.

[7] H. Guo, G. Wei, and J. An, “Dark Spot Detection in SAR Images of Oil Spill Using SegNet,” Applied Sciences, vol. 8, no. 12, p. 2670, Dec. 2018, doi: 10.3390/app8122670.

[8] O. Ronneberger, P. Fischer, and T. Brox, “U-Net: Convolutional Networks for Biomedical Image Segmentation,” 2015. Available: arxiv.org/pdf/1505.04597.pdf.

[9] D. Xiang, Y. Lu, D. Guan, G. Li, J. Cheng, and B. Li, “Oil Spill Detection in PolSAR Imagery Using Composite Scattering Power Entropy and Multi-Scale Hybrid Feature Fusion Network,” IEEE Journal of Selected Topics in Applied Earth Observations and Remote Sensing, pp. 1–20, Jan. 2025, doi: 10.1109/jstars.2025.3570697.

[10] R. M. Haralick, K. Shanmugam, and I. Dinstein, “Textural Features for Image Classification,” IEEE Transactions on Systems, Man, and Cybernetics, vol. SMC-3, no. 6, pp. 610–621, Nov. 1973, doi: 10.1109/tsmc.1973.4309314.

[11] A. S. Mahmoud, S. A. Mohamed, R. A. El-Khoriby, H. M. AbdelSalam, and I. A. El-Khodary, "Oil Spill Identification Based on Dual Attention U-Net Model Using Synthetic Aperture Radar Images," Journal of the Indian Society of Remote Sensing, vol. 51, no. 1, pp. 121–133, Nov. 2022, doi: 10.1007/s12524-022-01624-6.
[12] K.-F. Dagestad, J. Ro¨hrs, Ø. Breivik, and B. ˚Adlandsvik, "OpenDrift v1.0: A Generic Framework for Trajectory Modelling," Geoscientific Model Development, vol. 11, no. 4, pp. 1405–1420, Apr. 2018, doi: 10.5194/gmd-11- 1405-2018.
[13] M. De Dominicis, N. Pinardi, G. Zodiatis, and R. Lardner, "MEDSLIK-II: A Lagrangian Marine Surface Oil Spill Model for Short-Term Forecasting – Part 1: Theory," Geoscientific Model Development, vol. 6, no. 6, pp. 1851–1869, Nov. 2013, doi: 10.5194/gmd-6-1851-2013.
[14] D. Liu, Y. Li, and L. Mu, "Parameterization Modeling for Wind Drift Factor in Oil Spill Drift Trajectory Simulation Based on Machine Learning," Frontiers in Marine Science, vol. 10, Jul. 2023, doi: 10.3389/fmars.2023.1222347.
[15] B. G. Gautama, N. Longepe, R. Fablet, and G. Mercier, "Assimilative 2-D Lagrangian Transport Model for the Estimation of Oil Leakage Parameters from SAR Images: Application to the Montara Oil Spill," IEEE Journal of Selected Topics in Applied Earth Observations and Remote Sensing, vol. 9, no. 11, pp. 4962–4969, Nov. 2016, doi: 10.1109/jstars.2016.2606110.
[16] W. J. Guo and Y. X. Wang, "A Numerical Oil Spill Model Based on a Hybrid Method," Marine Pollution Bulletin, vol. 58, no. 5, pp. 726–734, May 2009, doi: 10.1016/j.marpolbul.2008.12.015.
[17] W. Shao et al., "Influence of Sea Surface Waves on Numerical Modeling of an Oil Spill: Revisit of Symphony Wheel Accident," Journal of Sea Research, vol. 201, pp. 102529–102529, Aug. 2024, doi: 10.1016/j.seares.2024.102529.

PRESENTATION CERTIFICATES

CERTIFICATE OF PRESENTATION & PUBLICATION

THE HINWEIS RESEARCH

Hinweis Fourth International Conference on Artificial Intelligence and Data Science, AIDE-2025

November 30, 2025

http://icaids.thehinweis.com/2025

Paras Ningune

Savitribai Phule Pune University, Pune, Maharashtra, India

Author of a Research Paper titled ***Oil Slick Detection and Drift Prediction using Satellite Imagery*** has submitted the paper which has been approved and presented for publication in the Hinweis Fourth International Conference on Artificial Intelligence and Data Science, AIDE-2025.

Dr. Yogesh Chaba
General Chair

Dr. Janahanlal Stephen
General Co Chair

CERTIFICATE OF PRESENTATION & PUBLICATION

THE HINWEIS RESEARCH

Hinweis Fourth International Conference on Artificial Intelligence and Data Science, AIDE-2025

November 30, 2025

http://icaids.thehinweis.com/2025

Sakshi Dilip Patil

Savitribai Phule Pune University, Pune, Maharashtra, India

Author of a Research Paper titled ***Oil Slick Detection and Drift Prediction using Satellite Imagery*** has submitted the paper which has been approved and presented for publication in the Hinweis Fourth International Conference on Artificial Intelligence and Data Science, AIDE-2025.

Dr. Yogesh Chaba
General Chair

Dr. Janahanlal Stephen
General Co Chair

CERTIFICATE OF PRESENTATION & PUBLICATION

THE HINWEIS RESEARCH

Hinweis Fourth International Conference on Artificial Intelligence and Data Science, AIDE-2025

November 30, 2025

http://icaids.thehinweis.com/2025

Soham Pramod Mane

Savitribai Phule Pune University, Pune, Maharashtra, India

Author of a Research Paper titled ***Oil Slick Detection and Drift Prediction using Satellite Imagery*** has submitted the paper which has been approved and presented for publication in the Hinweis Fourth International Conference on Artificial Intelligence and Data Science, AIDE-2025.

Dr. Yogesh Chaba
General Chair

Hinweis Research

Dr. Janahanlal Stephen
General Co Chair

CERTIFICATE OF PRESENTATION & PUBLICATION

THE HINWEIS RESEARCH

Hinweis Fourth International Conference on Artificial Intelligence and Data Science, AIDE-2025

November 30, 2025

http://icaids.thehinweis.com/2025

Siddhant Jagdish Pote

Savitribai Phule Pune University, Pune, Maharashtra, India

Author of a Research Paper titled ***Oil Slick Detection and Drift Prediction using Satellite Imagery*** has submitted the paper which has been approved and presented for publication in the Hinweis Fourth International Conference on Artificial Intelligence and Data Science, AIDE-2025.

Dr. Yogesh Chaba
General Chair

Hinweis Research

Dr. Janahanlal Stephen
General Co Chair